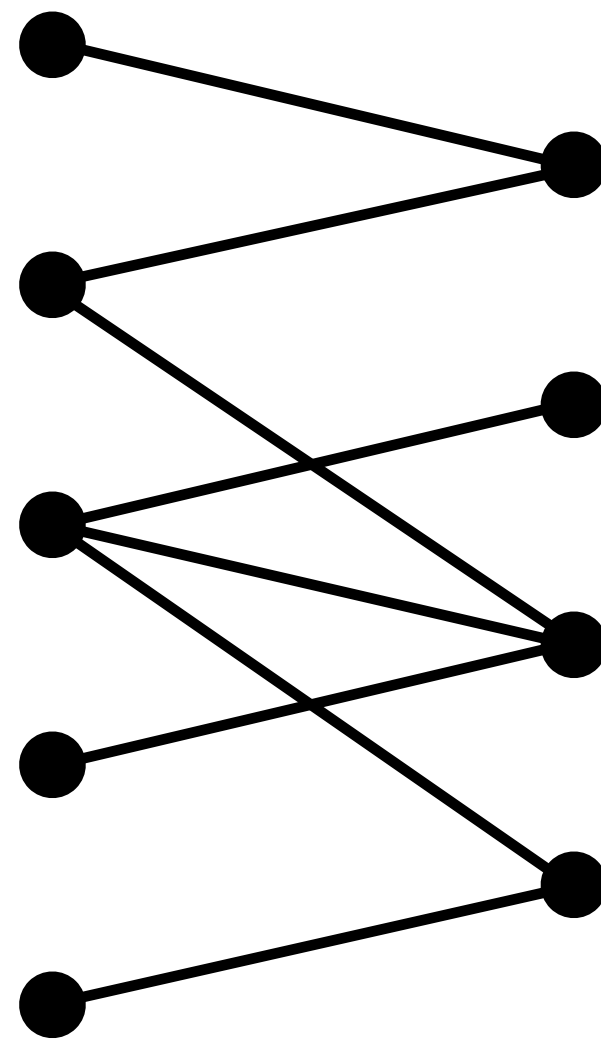


Lecture 21

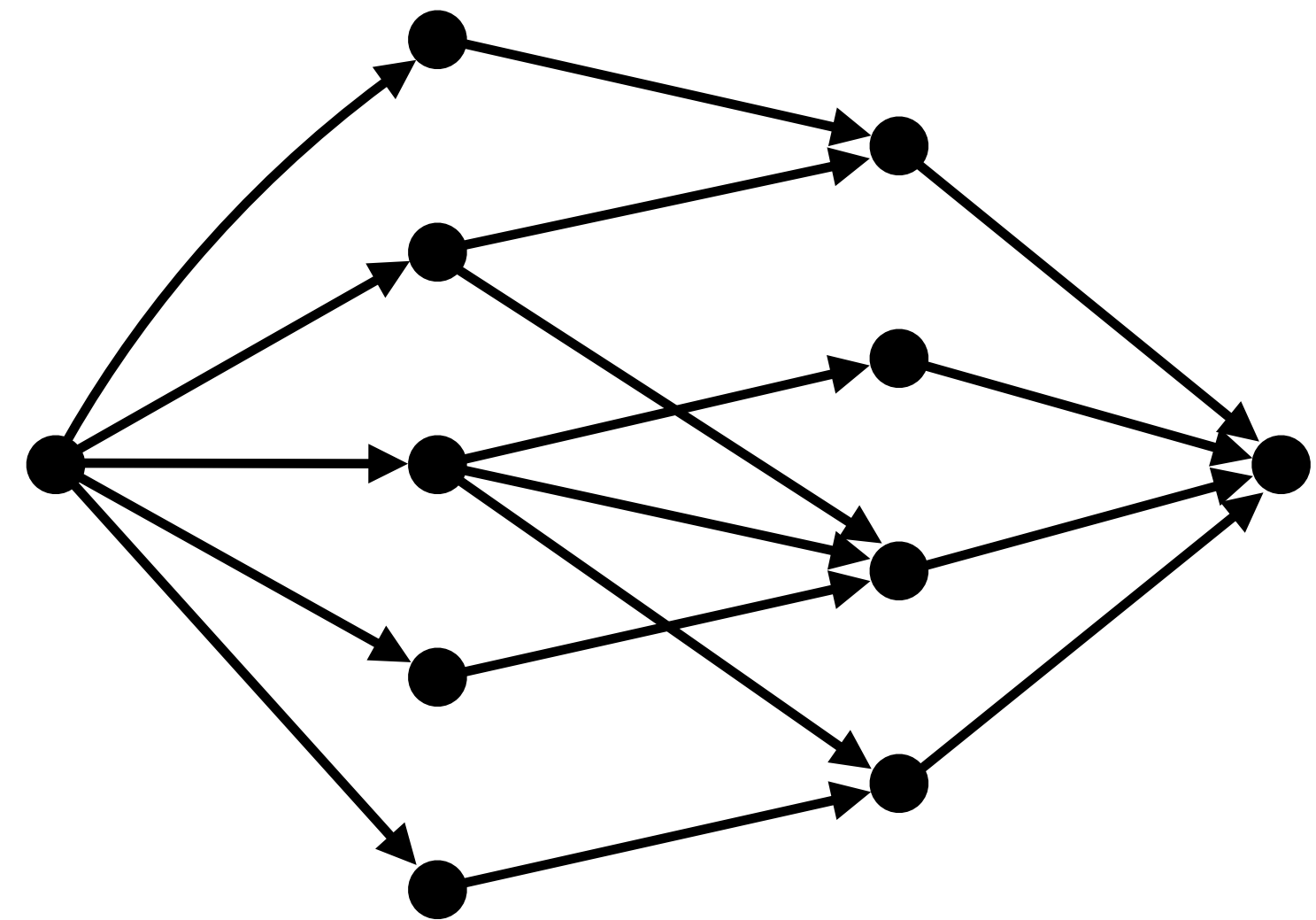
Bipartite Maximum Matching (contd.), Edmonds-Karp Algorithm

Solving *Max-Bipartite-Matching* **via** *Max-flow*

Solving *Max-Bipartite-Matching* via *Max-flow*

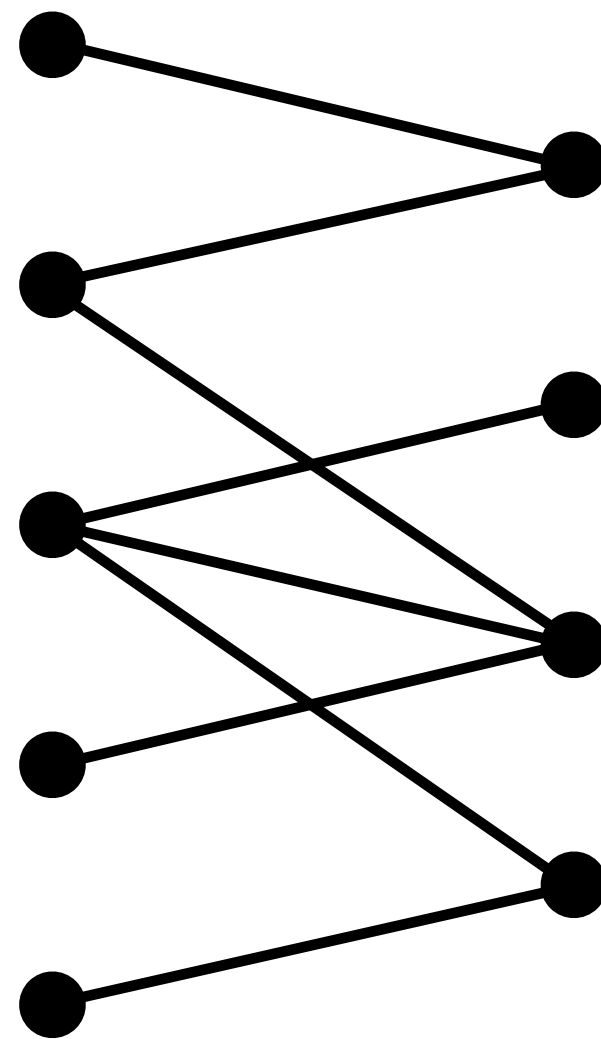


$$G = (V, E)$$



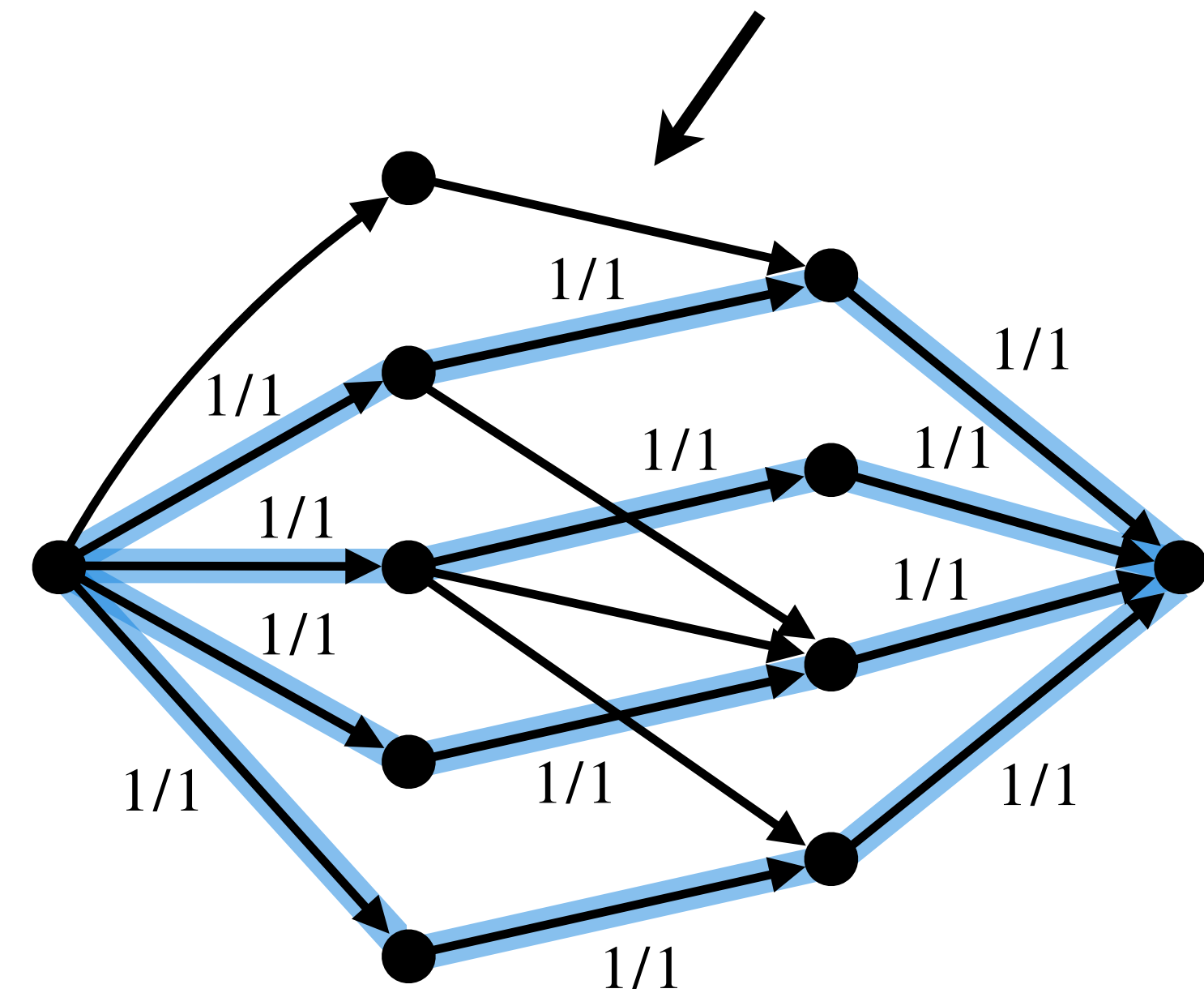
$$G' = (V', E')$$

Solving *Max-Bipartite-Matching* via *Max-flow*



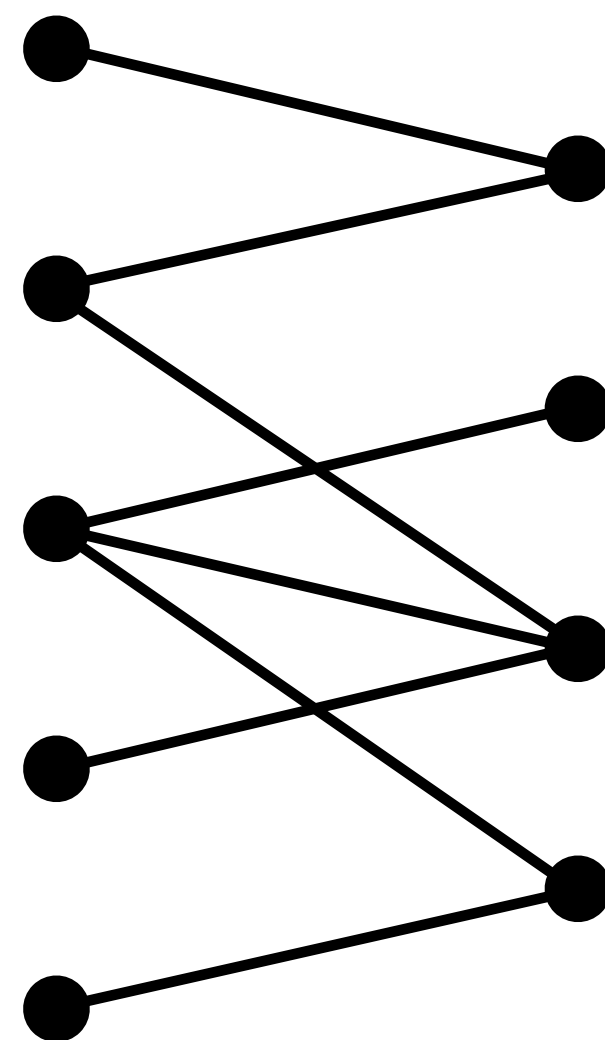
$$G = (V, E)$$

Integer-valued flow produced by Ford-Fulkerson



$$G' = (V', E')$$

Solving *Max-Bipartite-Matching* via *Max-flow*

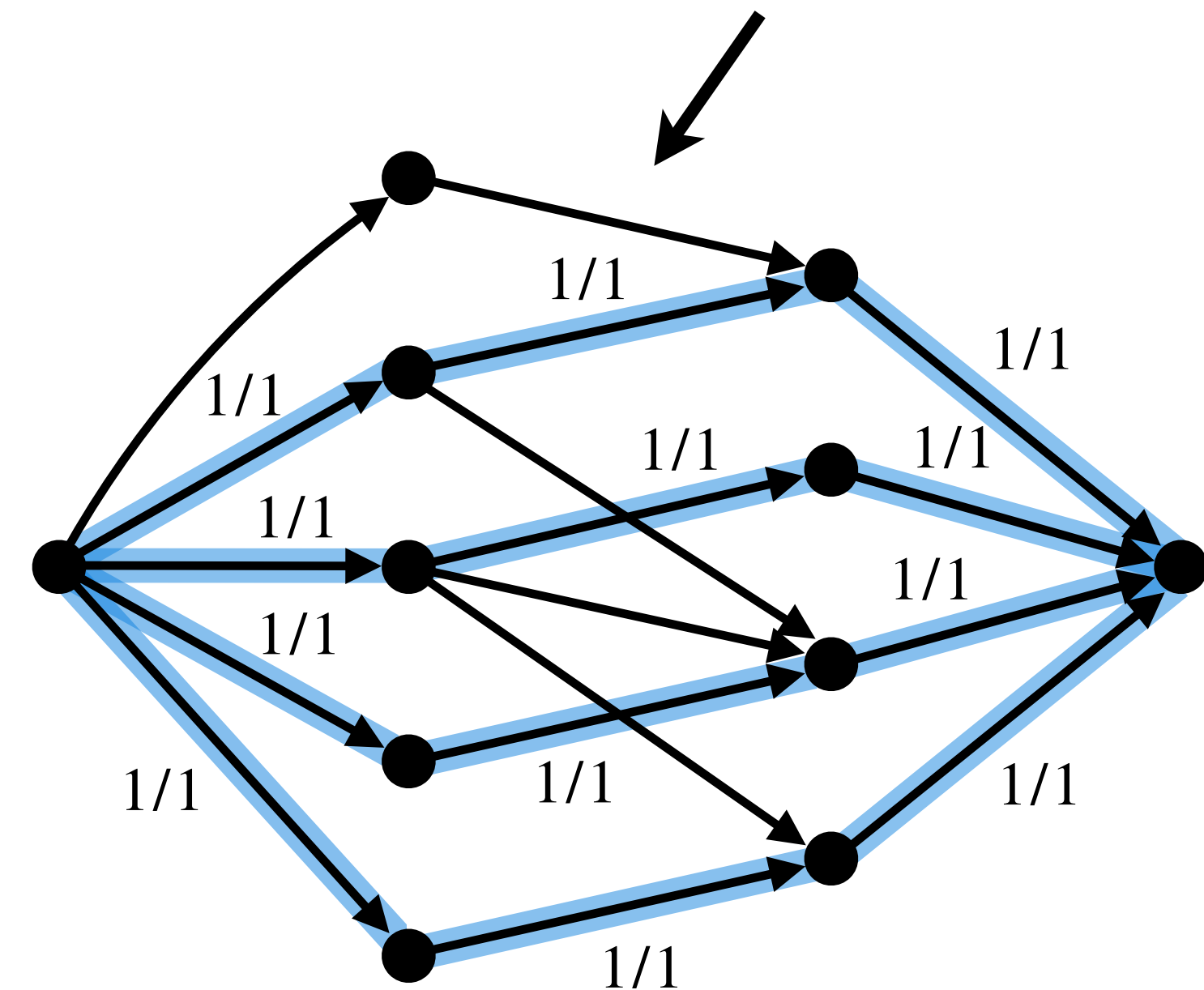


$$G = (V, E)$$

Corresponding matching

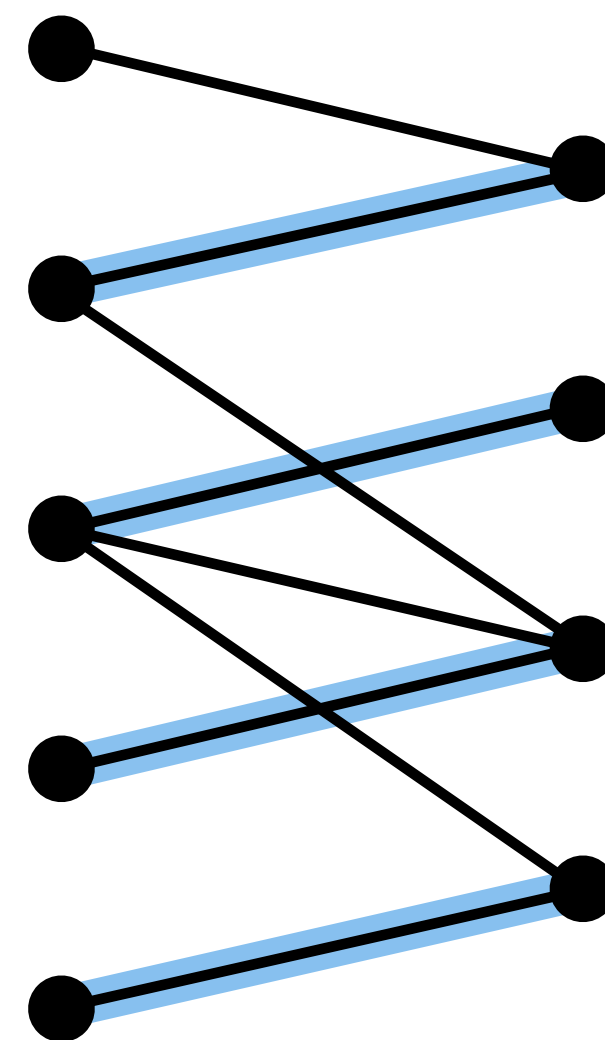


Integer-valued flow produced by Ford-Fulkerson



$$G' = (V', E')$$

Solving *Max-Bipartite-Matching* via *Max-flow*

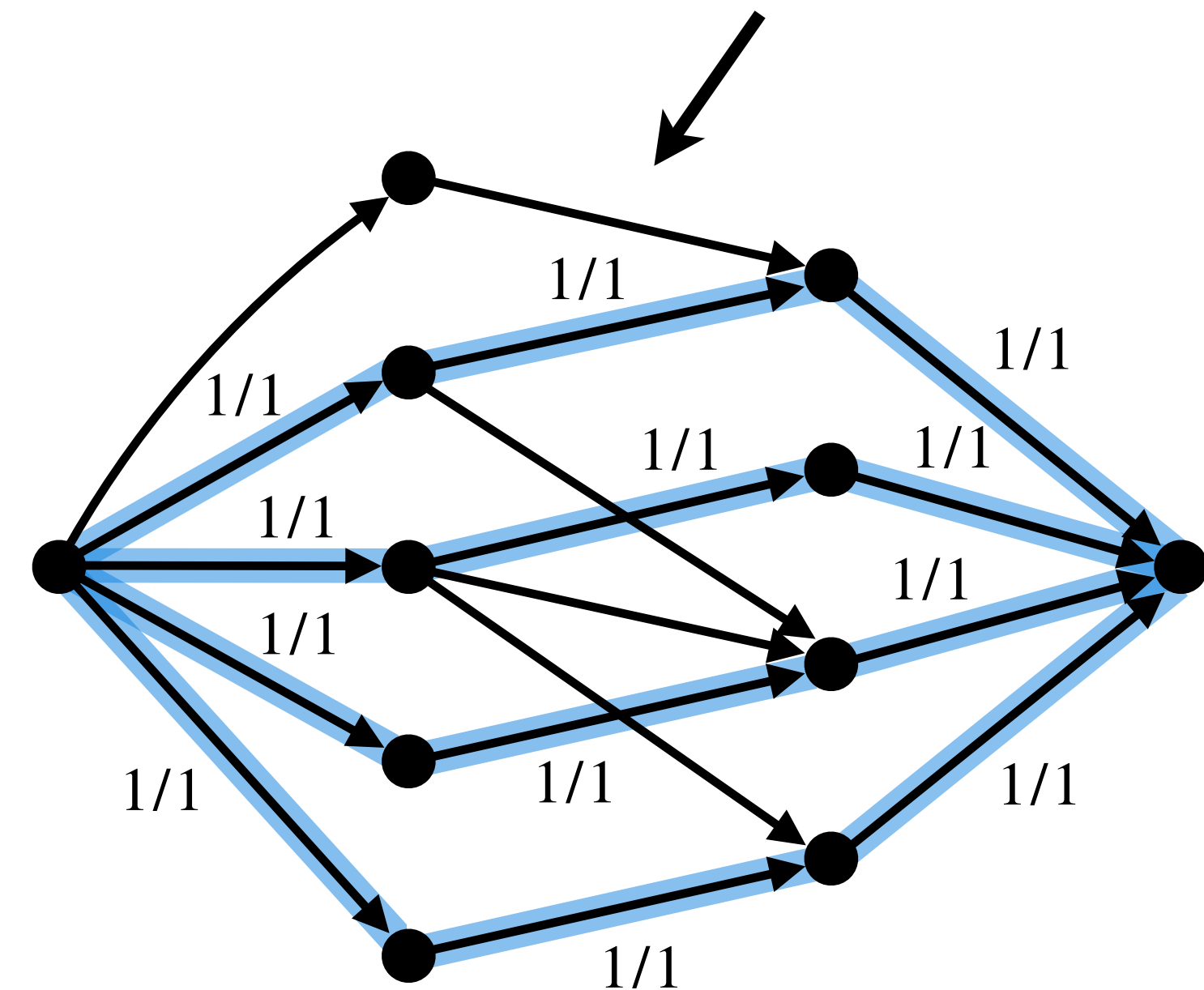


$$G = (V, E)$$

Corresponding matching



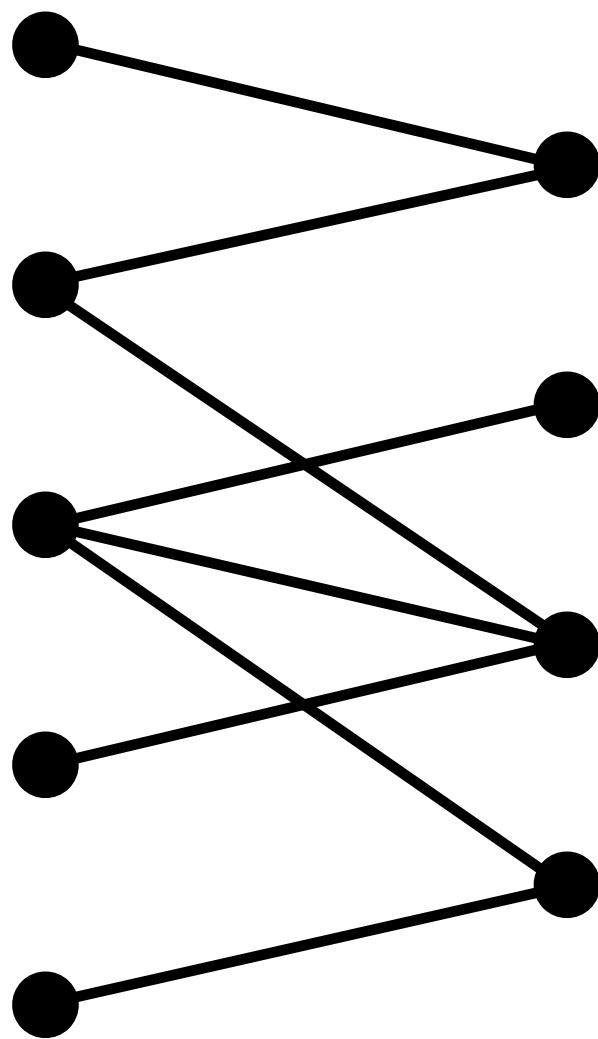
Integer-valued flow produced by Ford-Fulkerson



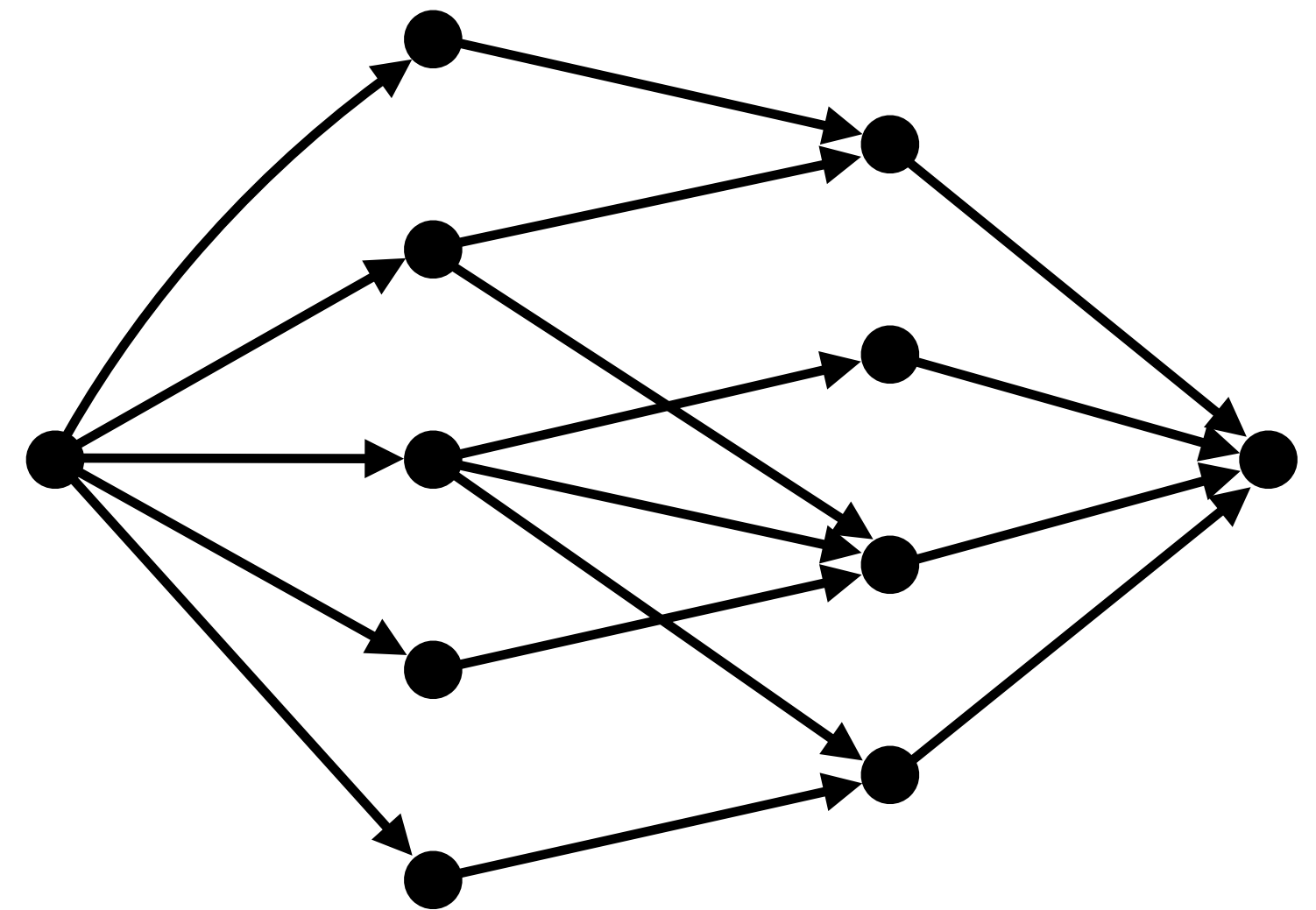
$$G' = (V', E')$$

Solving *Max-Bipartite-Matching* **via** *Max-flow*

Solving *Max-Bipartite-Matching* via *Max-flow*

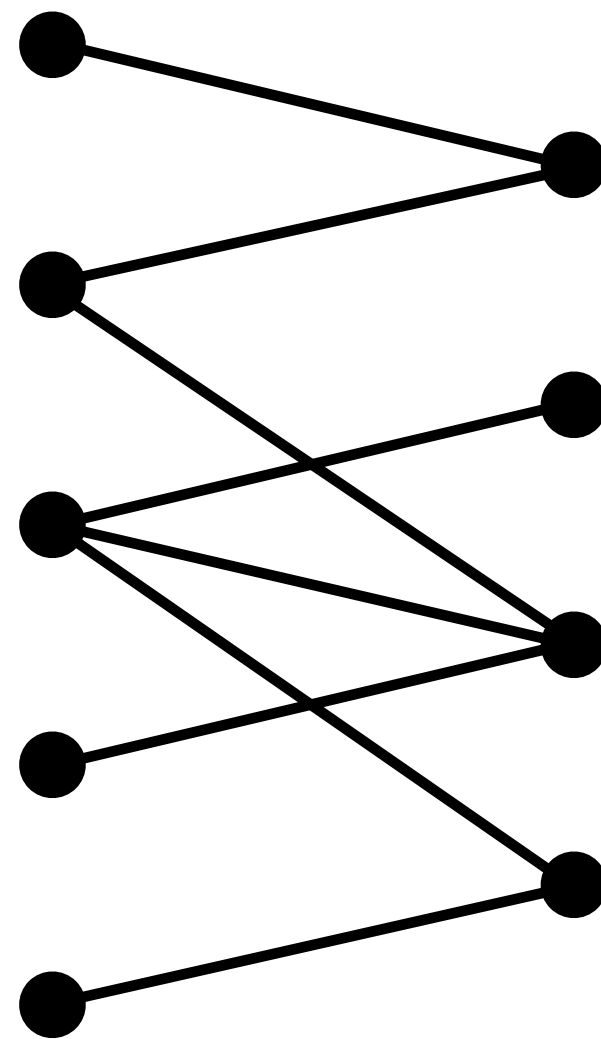


$$G = (V, E)$$

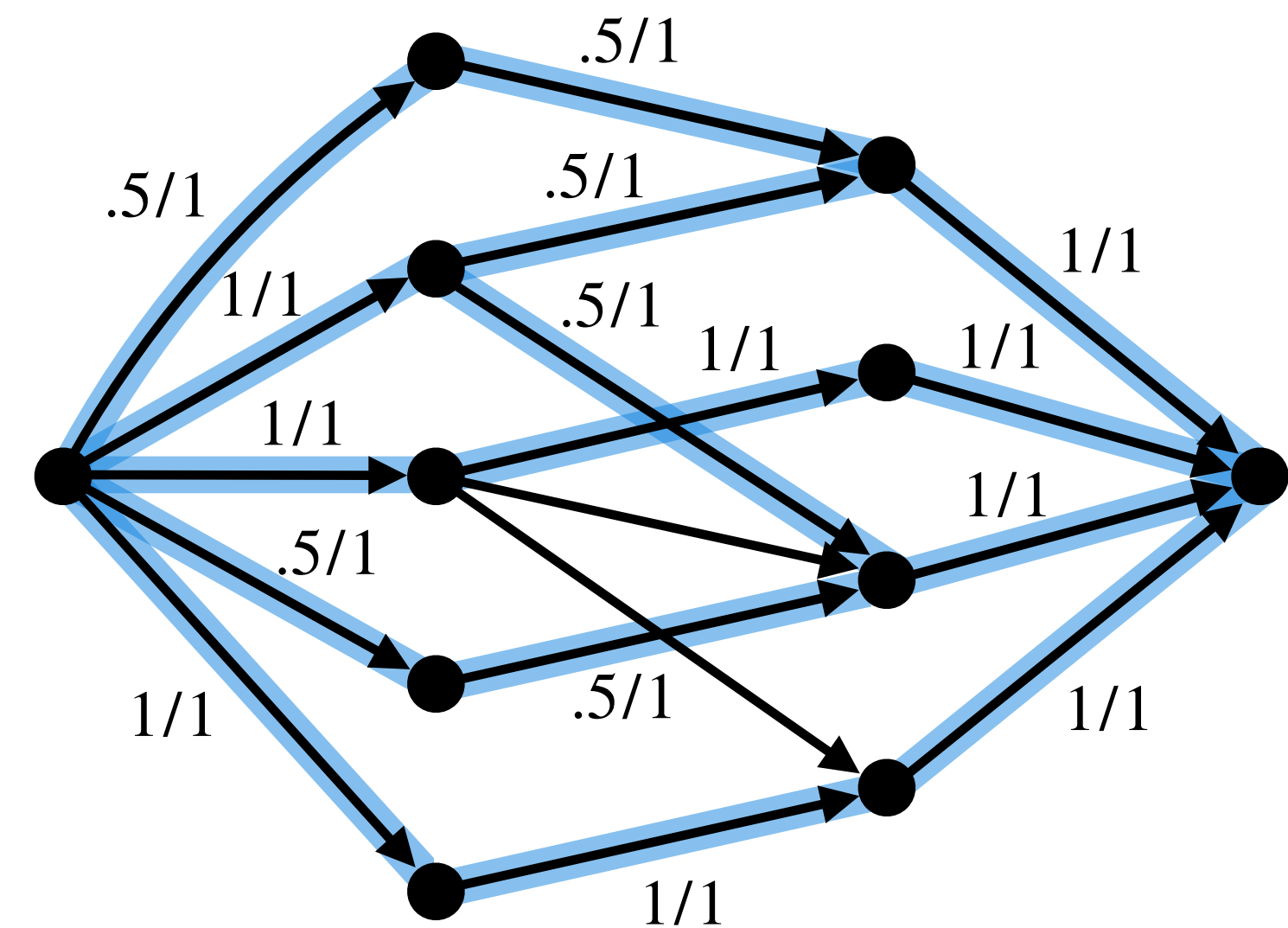


$$G' = (V', E')$$

Solving *Max-Bipartite-Matching* via *Max-flow*

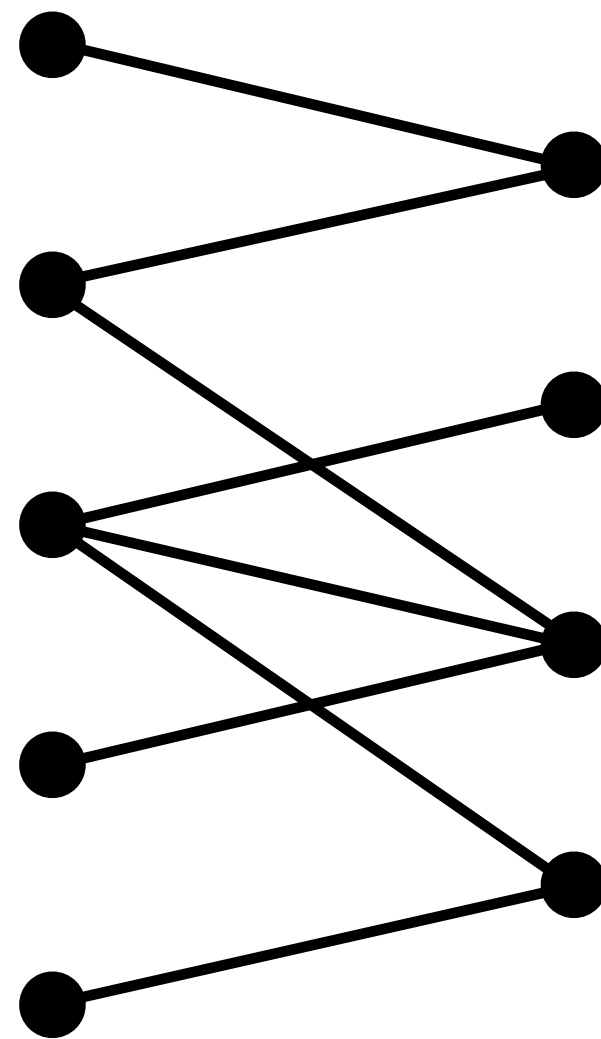


$$G = (V, E)$$



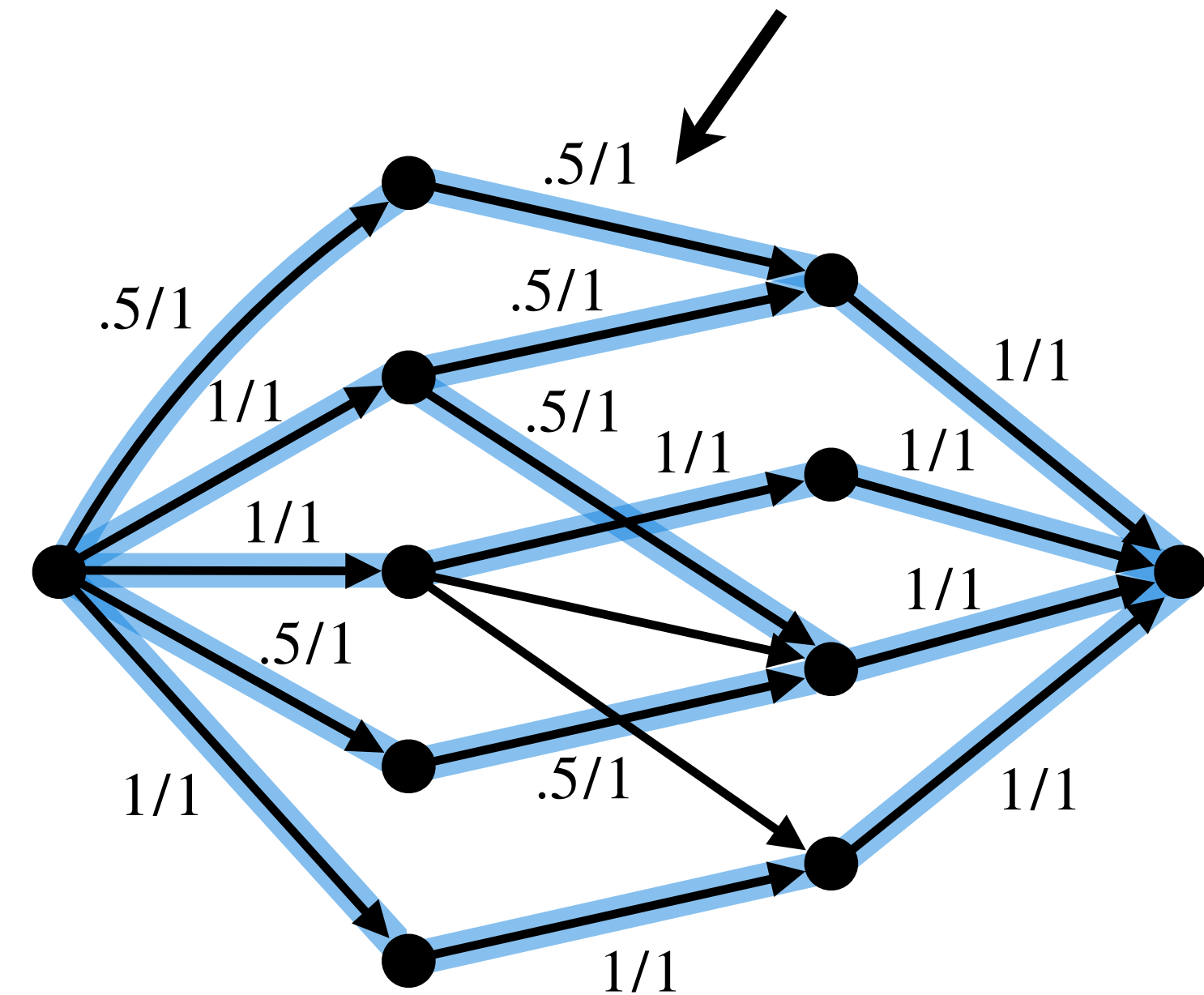
$$G' = (V', E')$$

Solving *Max-Bipartite-Matching* via *Max-flow*



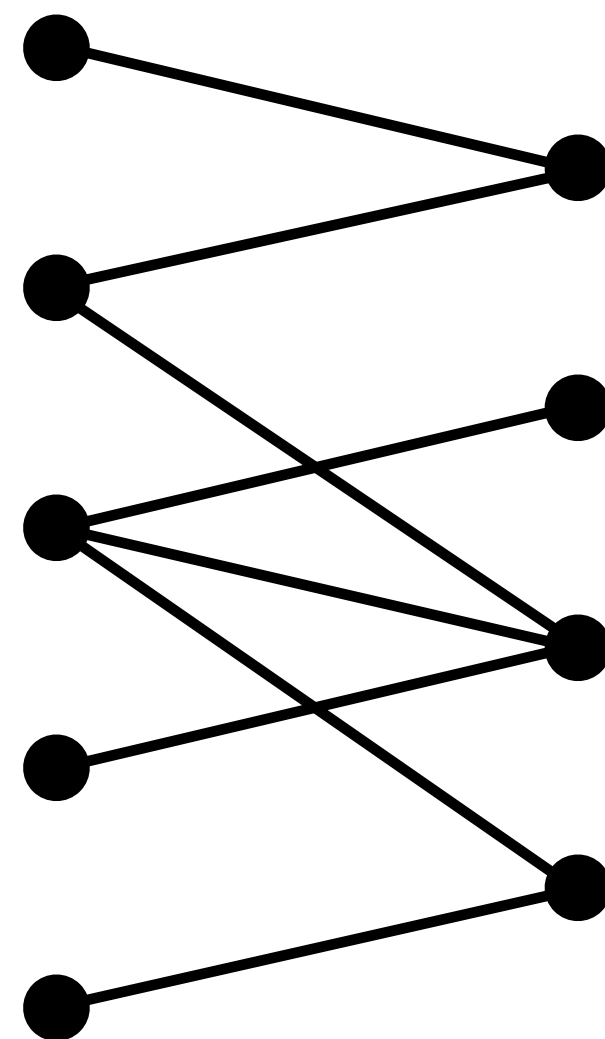
$$G = (V, E)$$

Fractional-flow produced by Ford-Fulkerson



$$G' = (V', E')$$

Solving *Max-Bipartite-Matching* via *Max-flow*

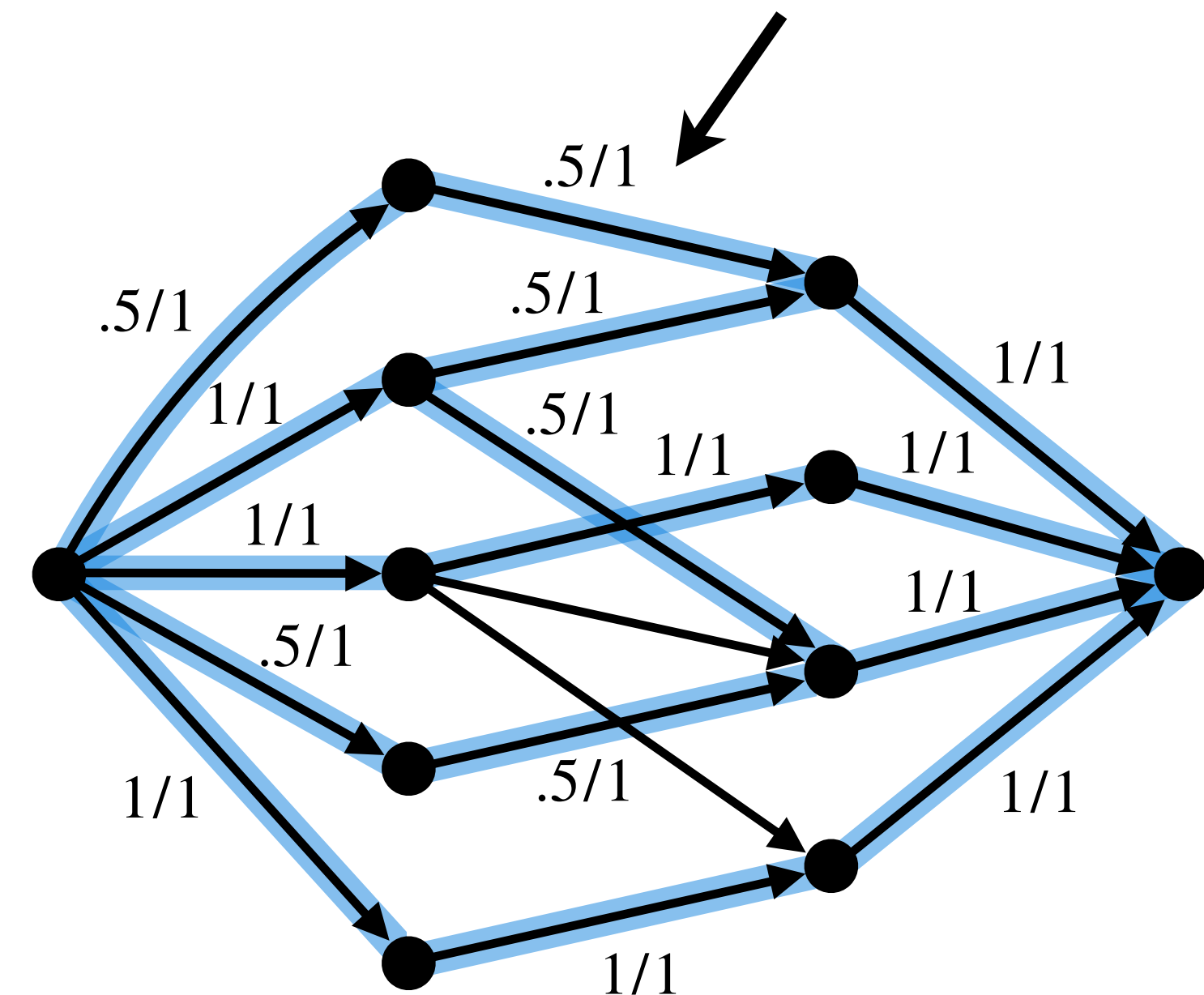


$$G = (V, E)$$

Corresponding matching?



Fractional-flow produced by Ford-Fulkerson



$$G' = (V', E')$$

Solving *Max-Bipartite-Matching* **via** *Max-flow*

Max-Bipartite-Matching(G):

1. Construct corresponding flow network G'
2. Find max-flow in G' using **Ford-Fulkerson**
3. **return** $\{(u, v) \mid u \in L, v \in R, f(u, v) = 1\}$

Solving *Max-Bipartite-Matching* via *Max-flow*

Max-Bipartite-Matching(G):

1. Construct corresponding flow network G'
2. Find max-flow in G' using **Ford-Fulkerson**
3. **return** $\{(u, v) \mid u \in L, v \in R, f(u, v) = 1\}$

Correctness: Max-flow produced using **Ford-Fulkerson** method on a network with integer

Solving *Max-Bipartite-Matching* via *Max-flow*

Max-Bipartite-Matching(G):

1. Construct corresponding flow network G'
2. Find max-flow in G' using **Ford-Fulkerson**
3. **return** $\{(u, v) \mid u \in L, v \in R, f(u, v) = 1\}$

Correctness: Max-flow produced using **Ford-Fulkerson** method on a network with integer capacities is **integer-valued**.

Solving *Max-Bipartite-Matching* via *Max-flow*

Max-Bipartite-Matching(G):

1. Construct corresponding flow network G'
2. Find max-flow in G' using **Ford-Fulkerson**
3. **return** $\{(u, v) \mid u \in L, v \in R, f(u, v) = 1\}$

Correctness: Max-flow produced using **Ford-Fulkerson** method on a network with integer capacities is **integer-valued**. Remaining proof follows from the previous claim.

Solving *Max-Bipartite-Matching* **via** *Max-flow*

Max-Bipartite-Matching(G):

1. Construct corresponding flow network G'
2. Find max-flow in G' using **Ford-Fulkerson**
3. **return** $\{(u, v) \mid u \in L, v \in R, f(u, v) = 1\}$

Solving *Max-Bipartite-Matching* via *Max-flow*

Max-Bipartite-Matching(G):

1. Construct corresponding flow network G'
2. Find max-flow in G' using **Ford-Fulkerson**
3. **return** $\{(u, v) \mid u \in L, v \in R, f(u, v) = 1\}$

$\Theta(V + E)$ time to find partition (L, R)
and add $|V|$ many new edges are.



Solving *Max-Bipartite-Matching* via *Max-flow*

Max-Bipartite-Matching(G):

1. Construct corresponding flow network G'
2. Find max-flow in G' using **Ford-Fulkerson**
3. **return** $\{(u, v) \mid u \in L, v \in R, f(u, v) = 1\}$

$\Theta(V + E)$ time to find partition (L, R)
and add $|V|$ many new edges are.

$\Theta(|E| \cdot |f^*|)$

Solving *Max-Bipartite-Matching* via *Max-flow*

Max-Bipartite-Matching(G):

1. Construct corresponding flow network G'
2. Find max-flow in G' using **Ford-Fulkerson**
3. **return** $\{(u, v) \mid u \in L, v \in R, f(u, v) = 1\}$

$\Theta(V + E)$ time to find partition (L, R)
and add $|V|$ many new edges are.

$\Theta(|E| \cdot |f^*|)$

Time Complexity: $\Theta(|E| \cdot |f^*|)$, when $|E| \geq |V|$.

Solving Max-Bipartite-Matching via Max-flow

Max-Bipartite-Matching(G):

1. Construct corresponding flow network G'
2. Find max-flow in G' using **Ford-Fulkerson**
3. **return** $\{(u, v) \mid u \in L, v \in R, f(u, v) = 1\}$

$\Theta(V + E)$ time to find partition (L, R)
and add $|V|$ many new edges are.

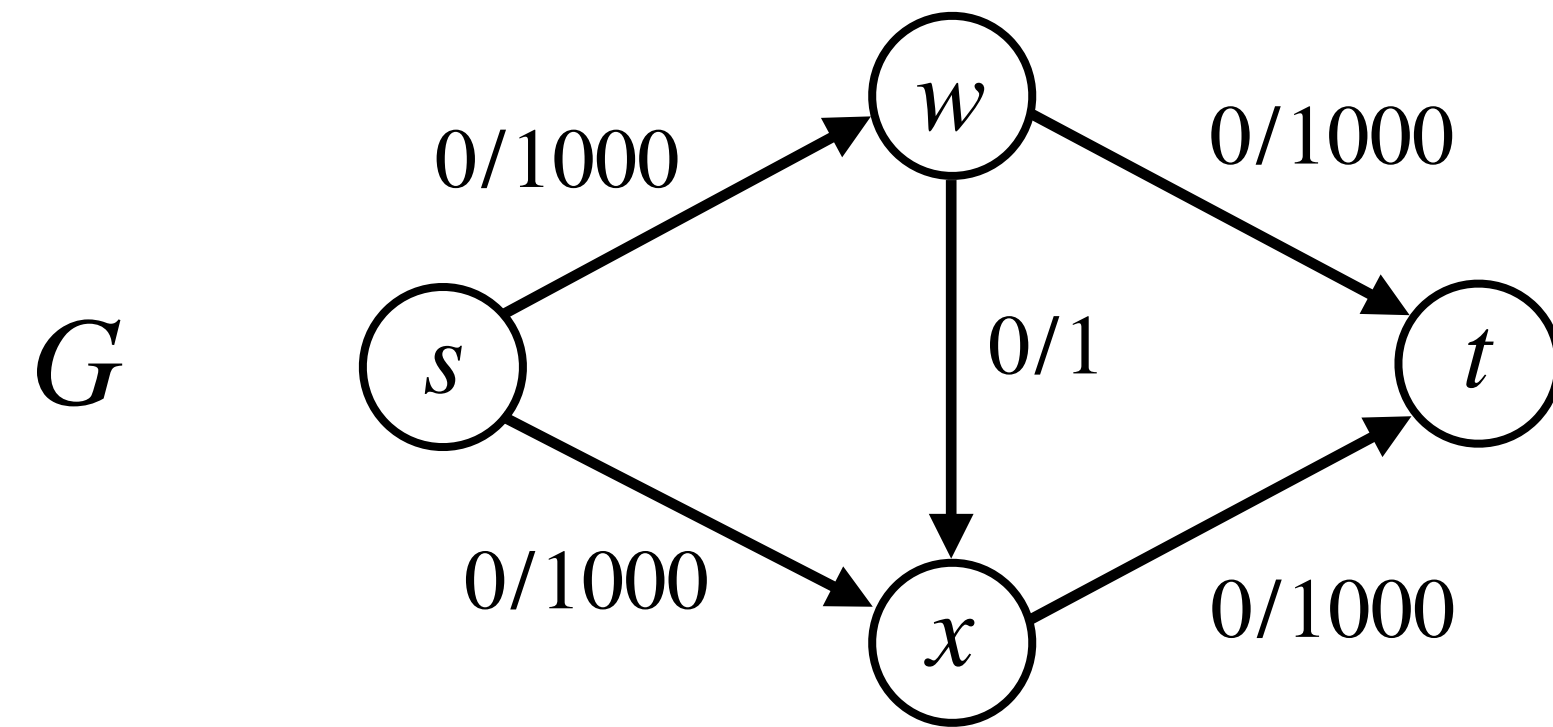
$\Theta(|E| \cdot |f^*|)$

Time Complexity: $\Theta(|E| \cdot |f^*|)$, when $|E| \geq |V|$.

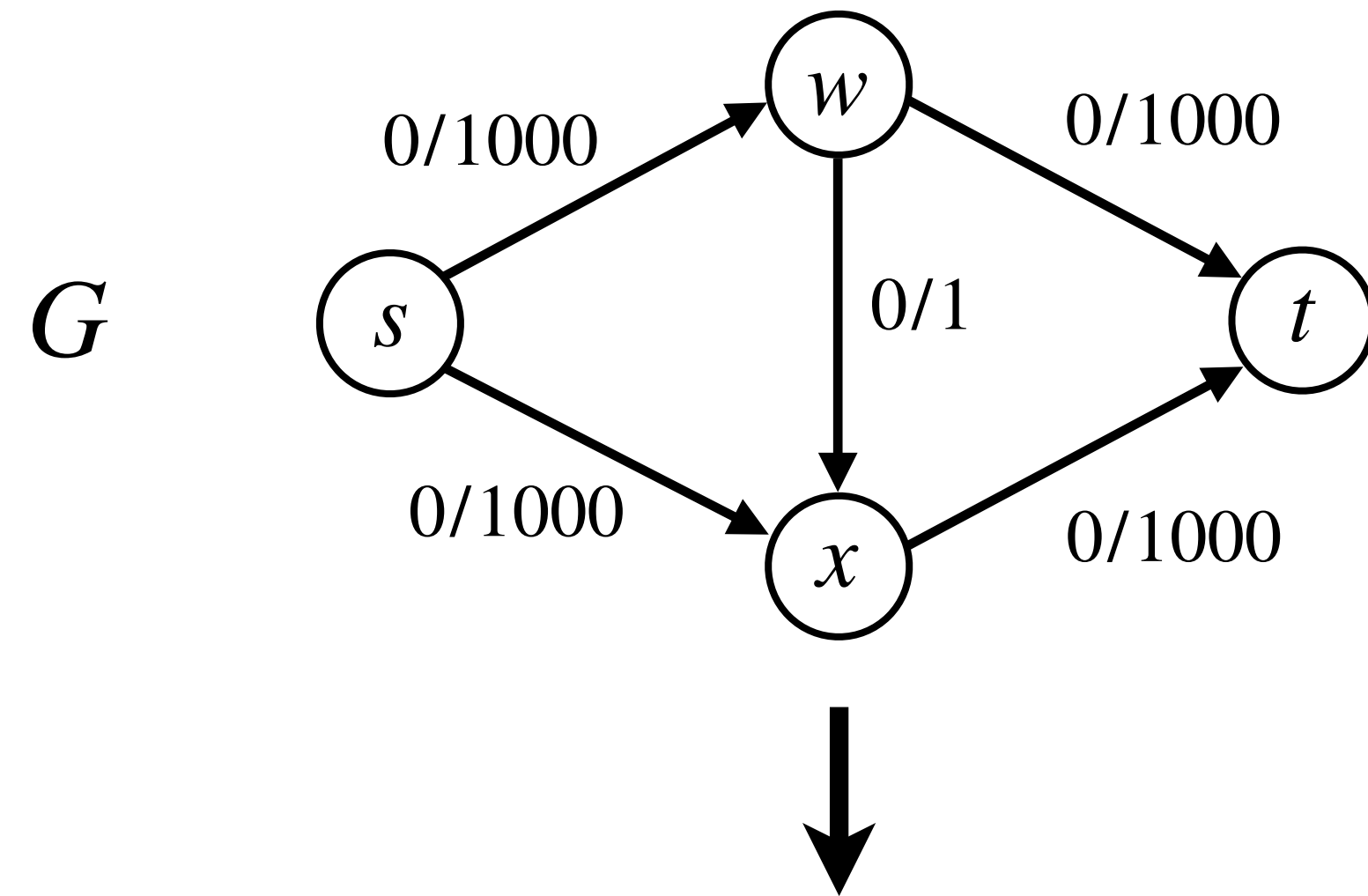
$$|f^*| \leq |E|$$

A Bad Case for Ford-Fulkerson

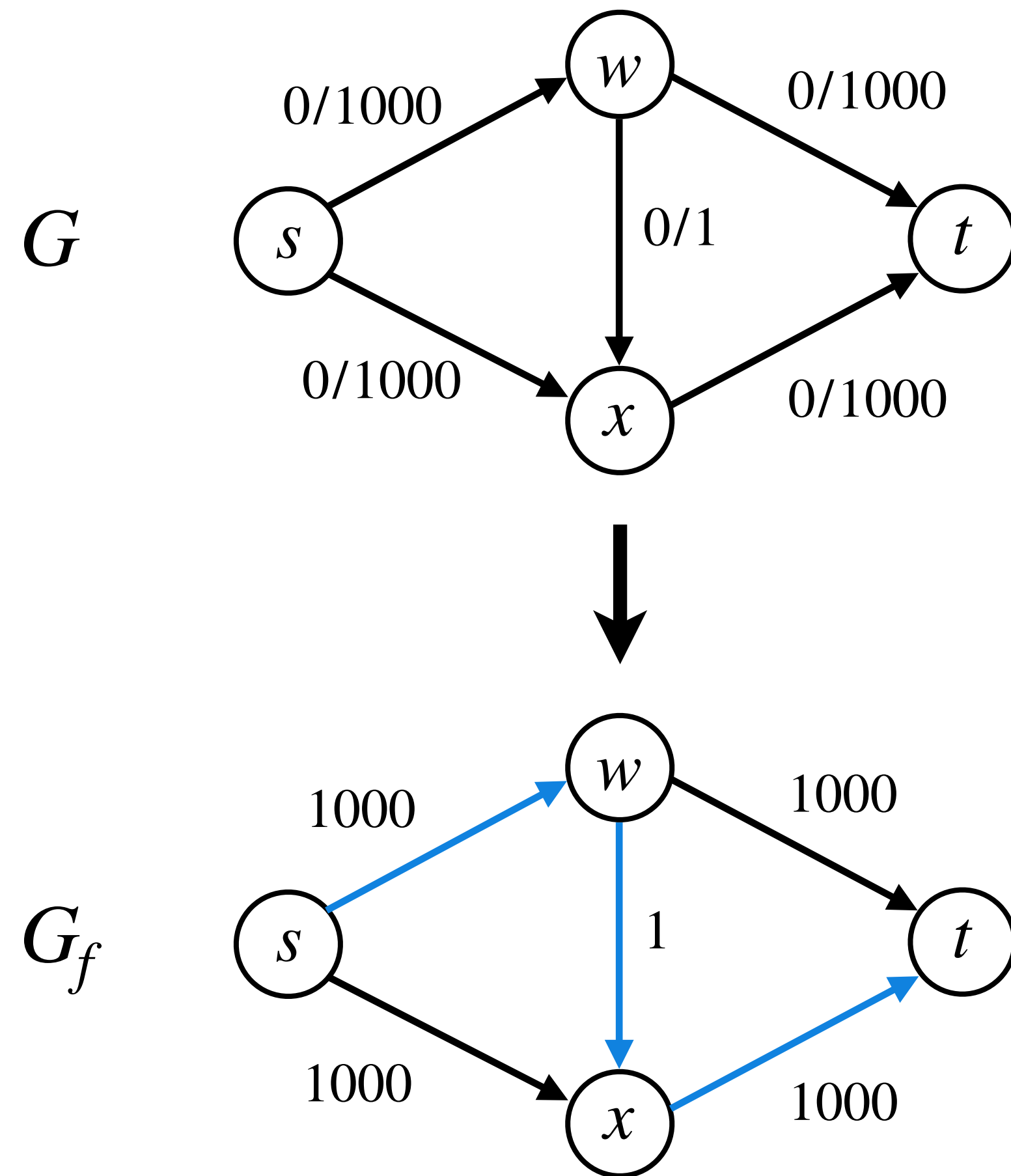
A Bad Case for Ford-Fulkerson



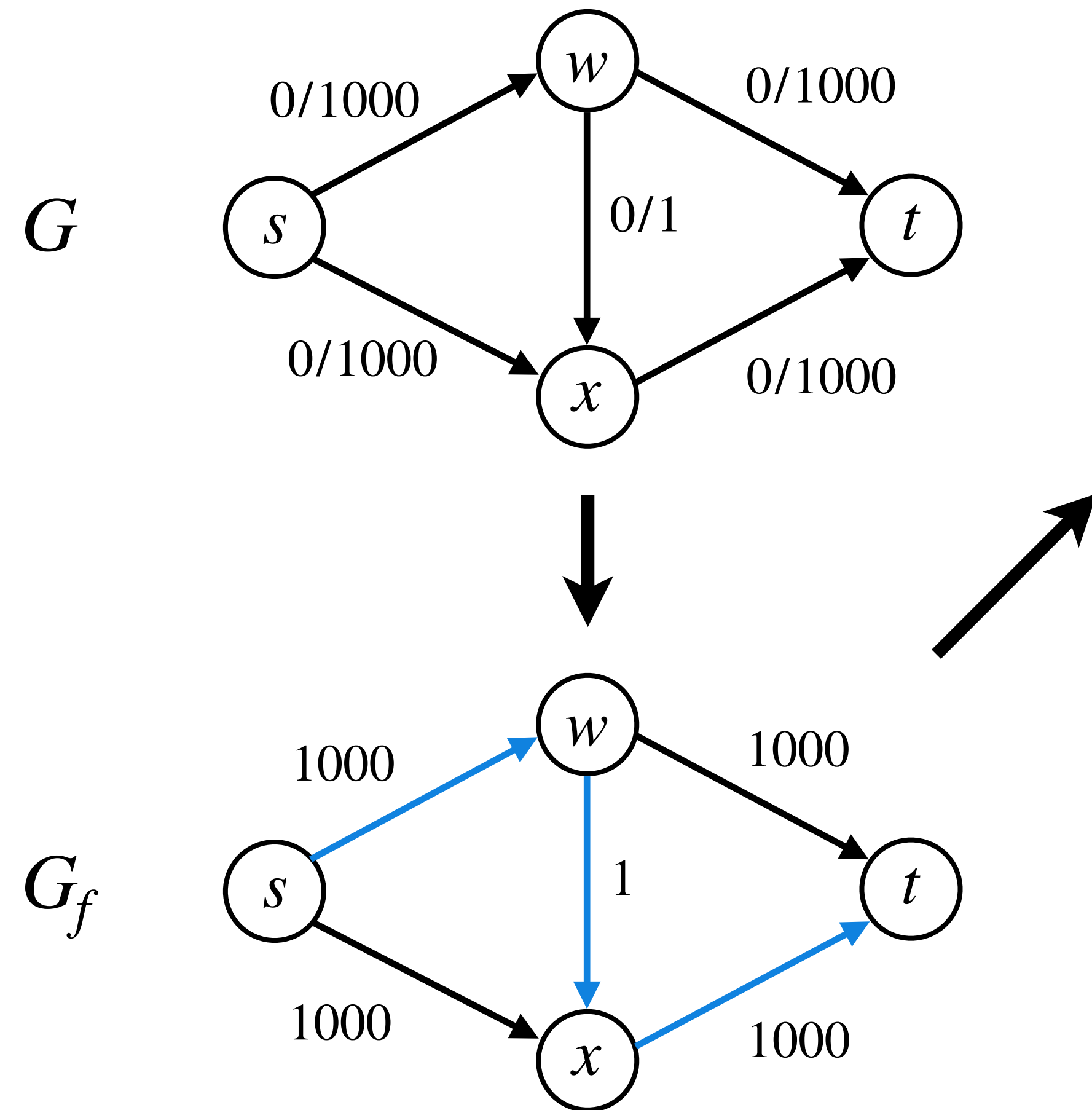
A Bad Case for Ford-Fulkerson



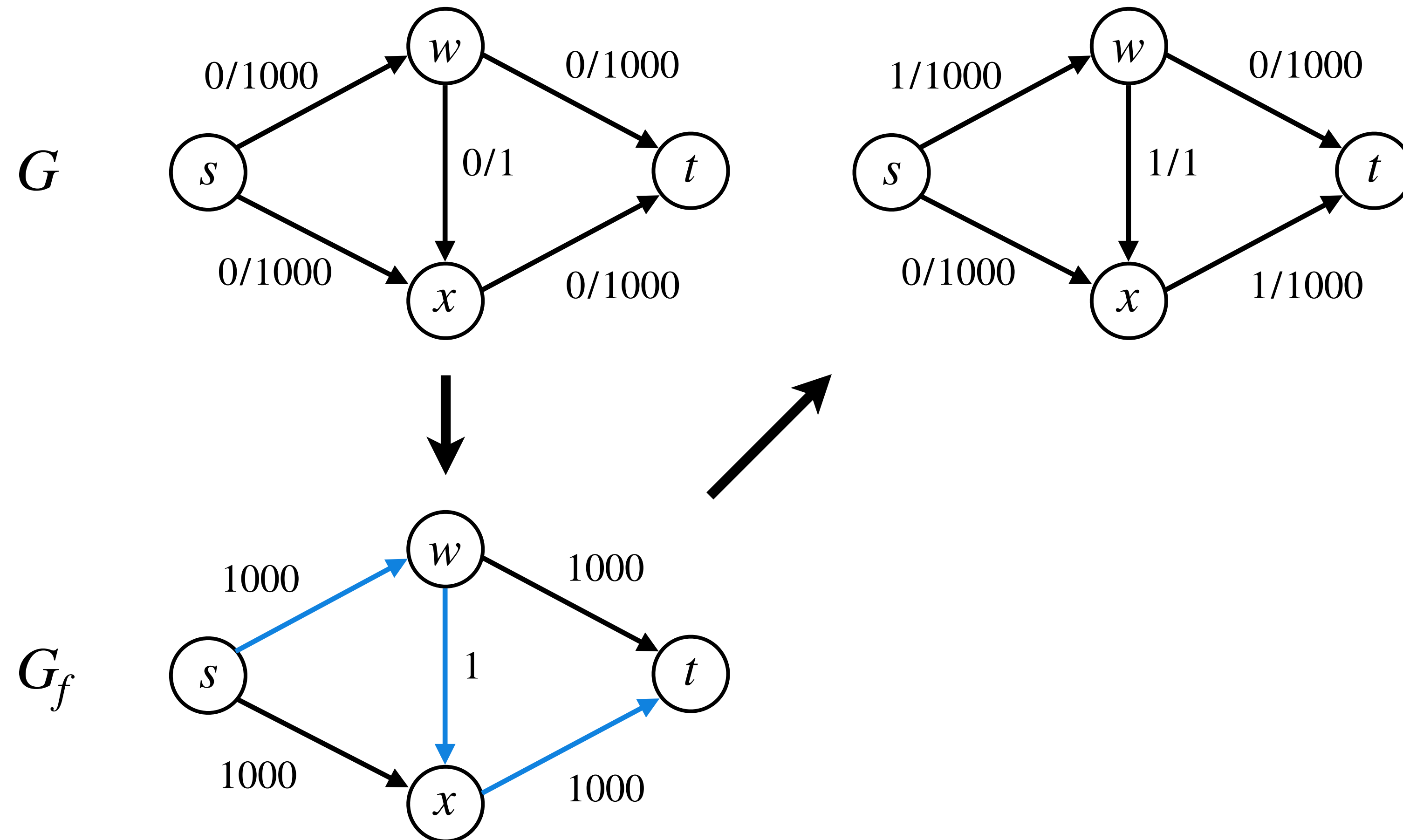
A Bad Case for Ford-Fulkerson



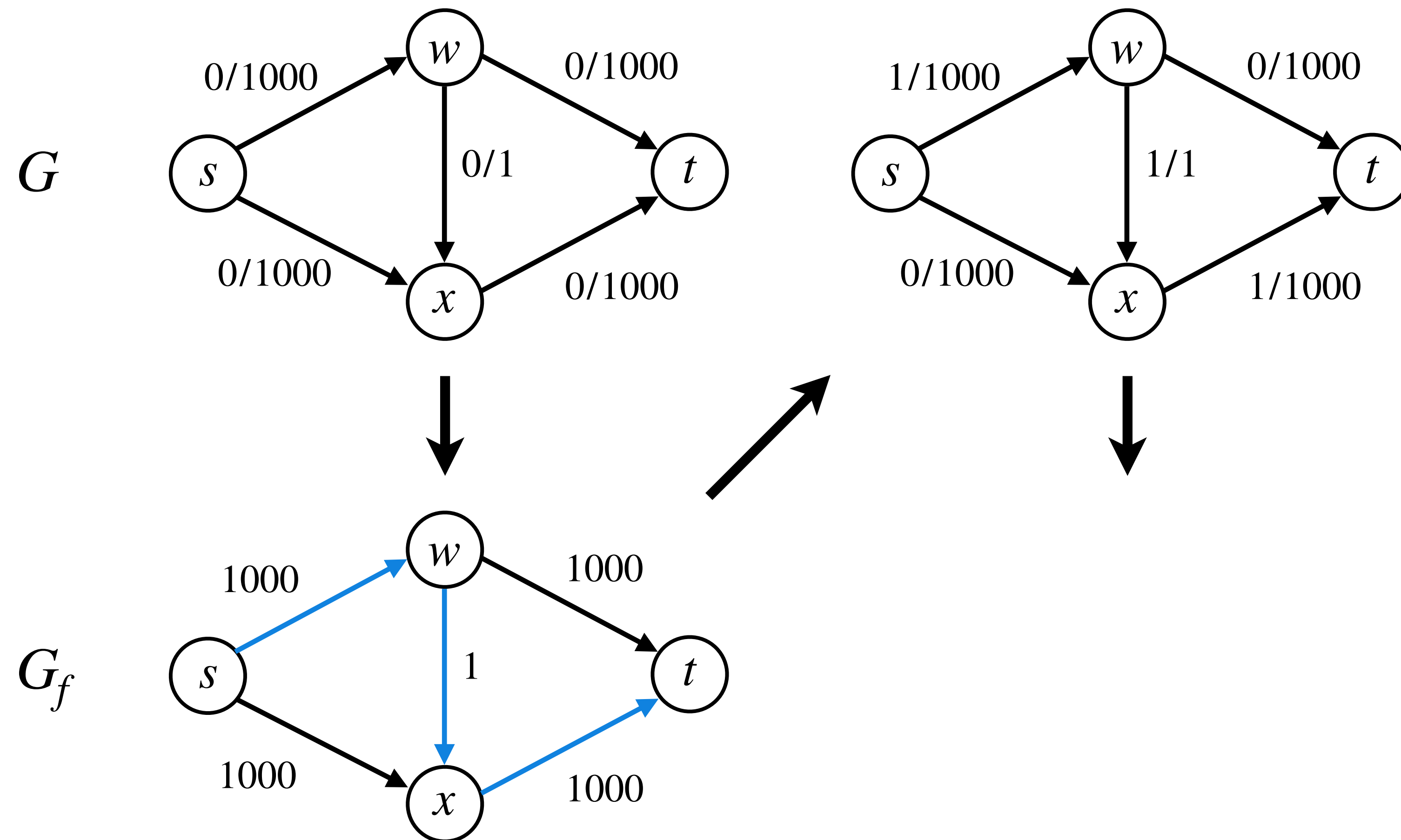
A Bad Case for Ford-Fulkerson



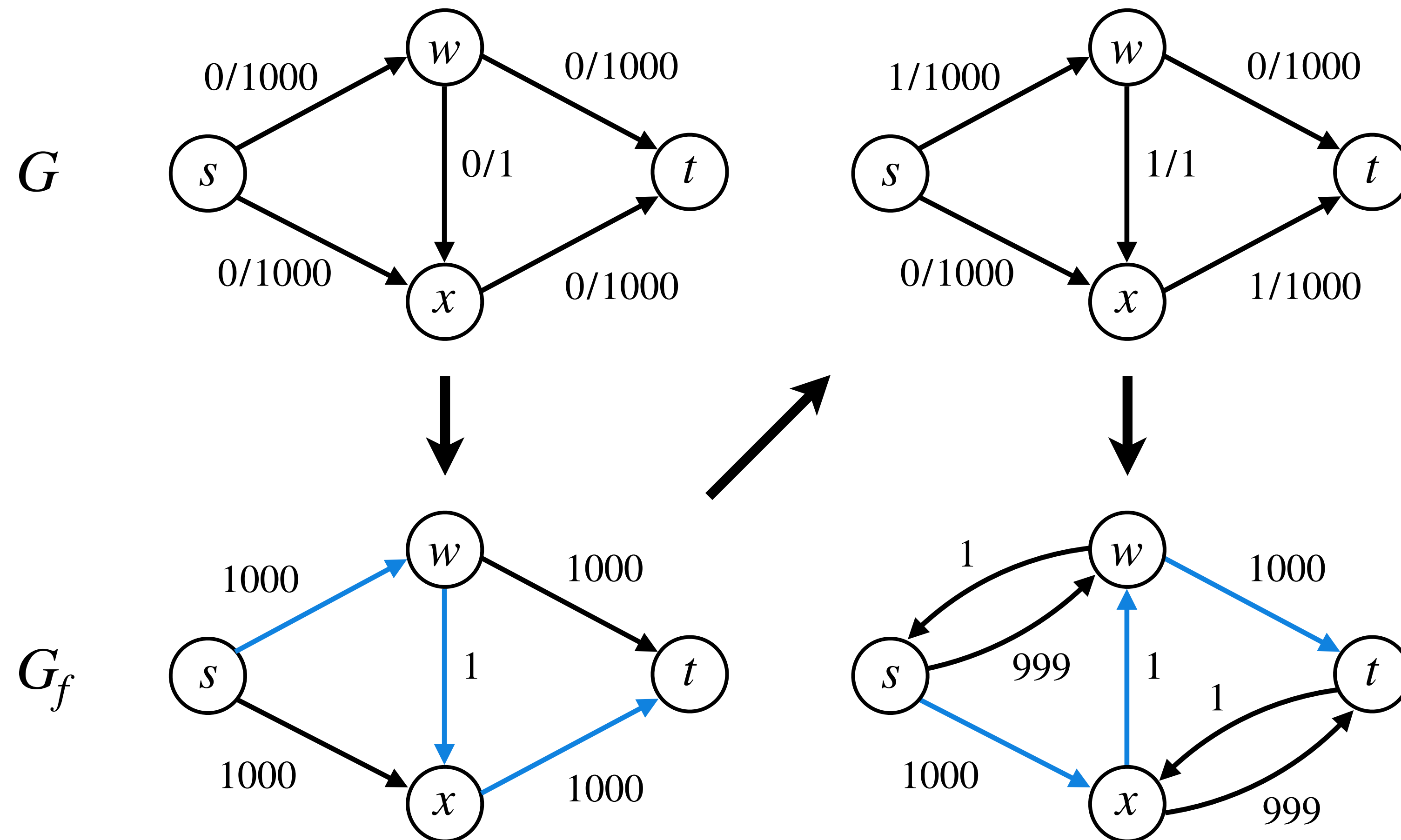
A Bad Case for Ford-Fulkerson



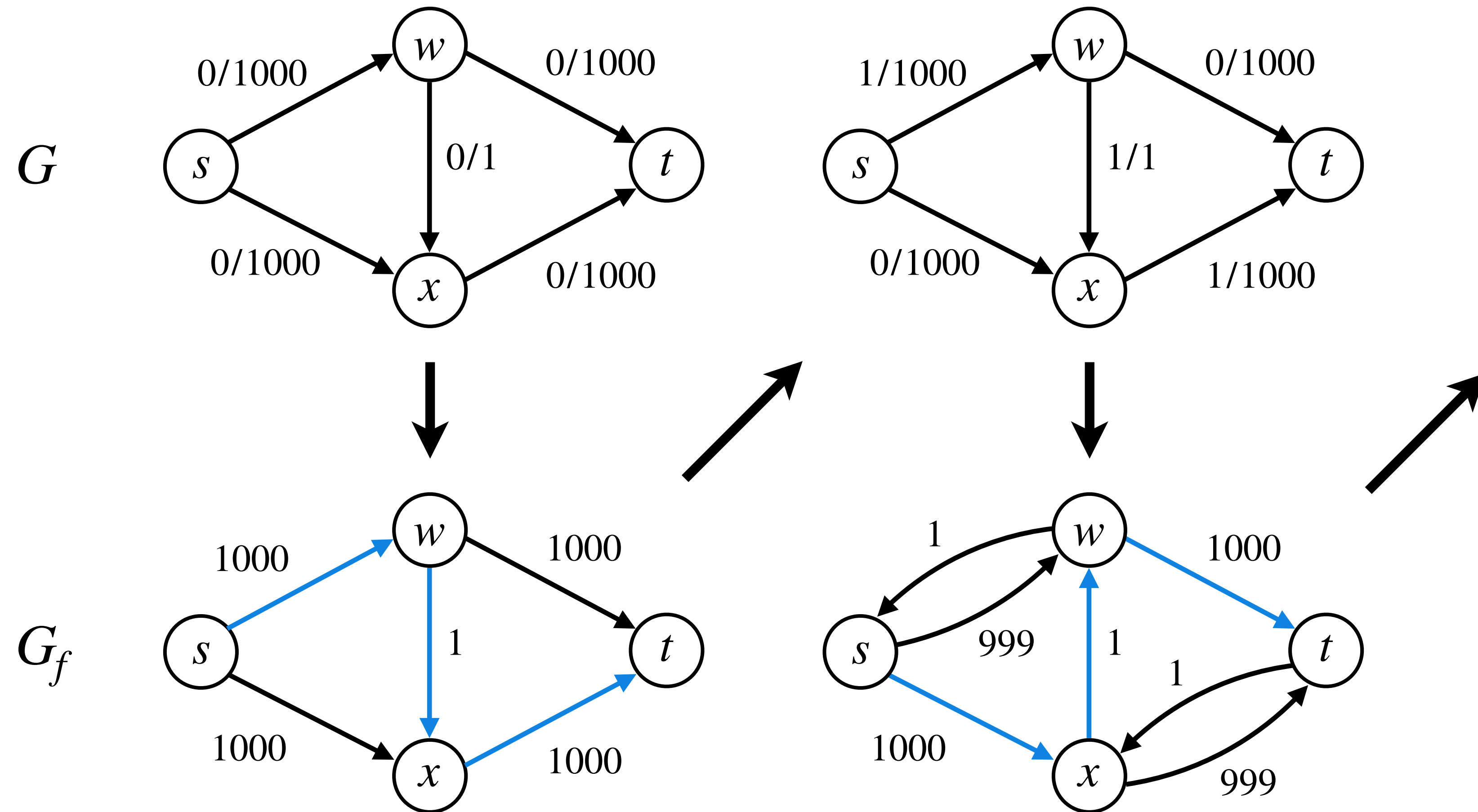
A Bad Case for Ford-Fulkerson



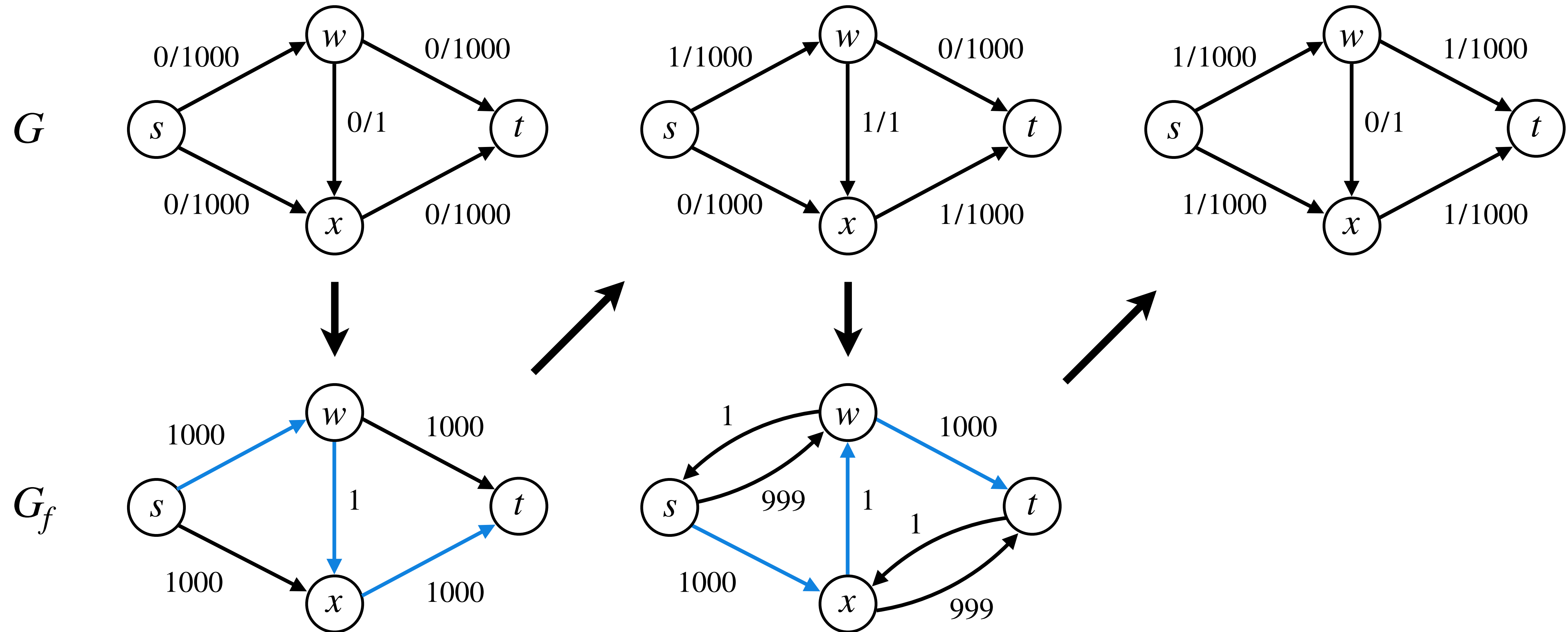
A Bad Case for Ford-Fulkerson



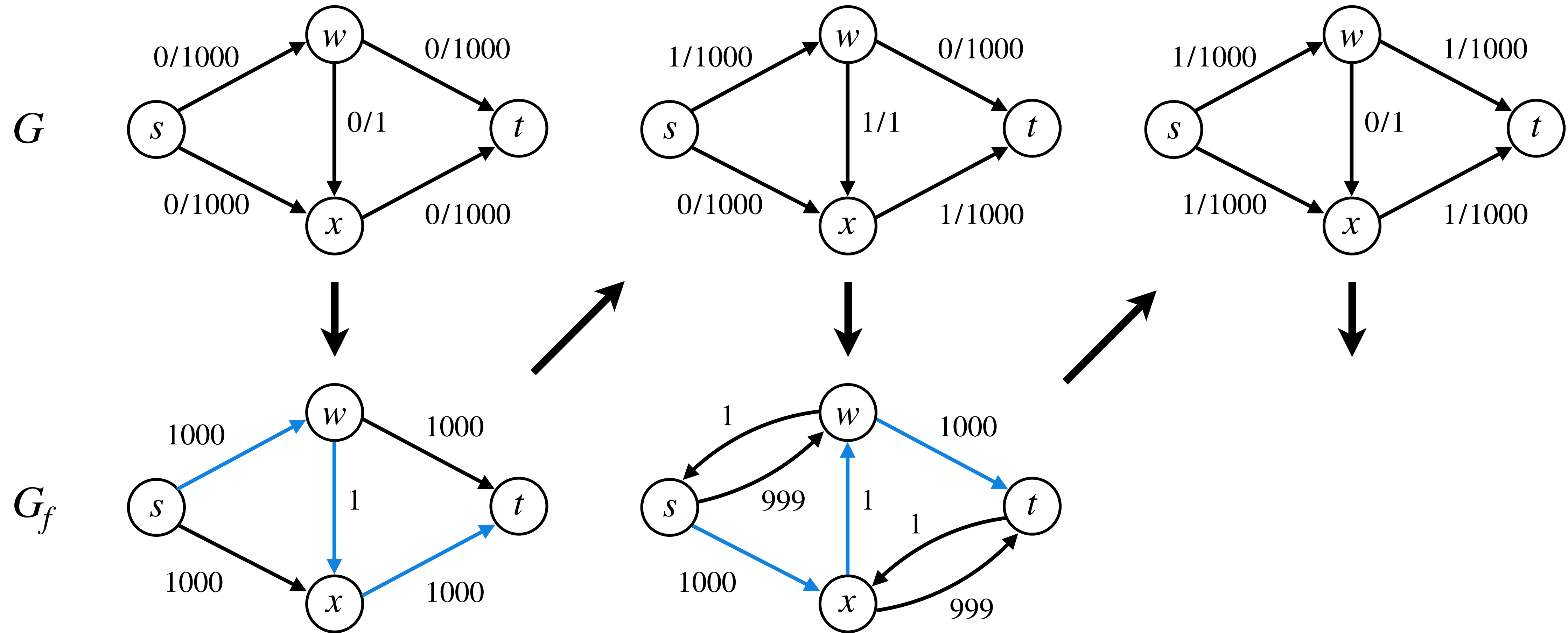
A Bad Case for Ford-Fulkerson



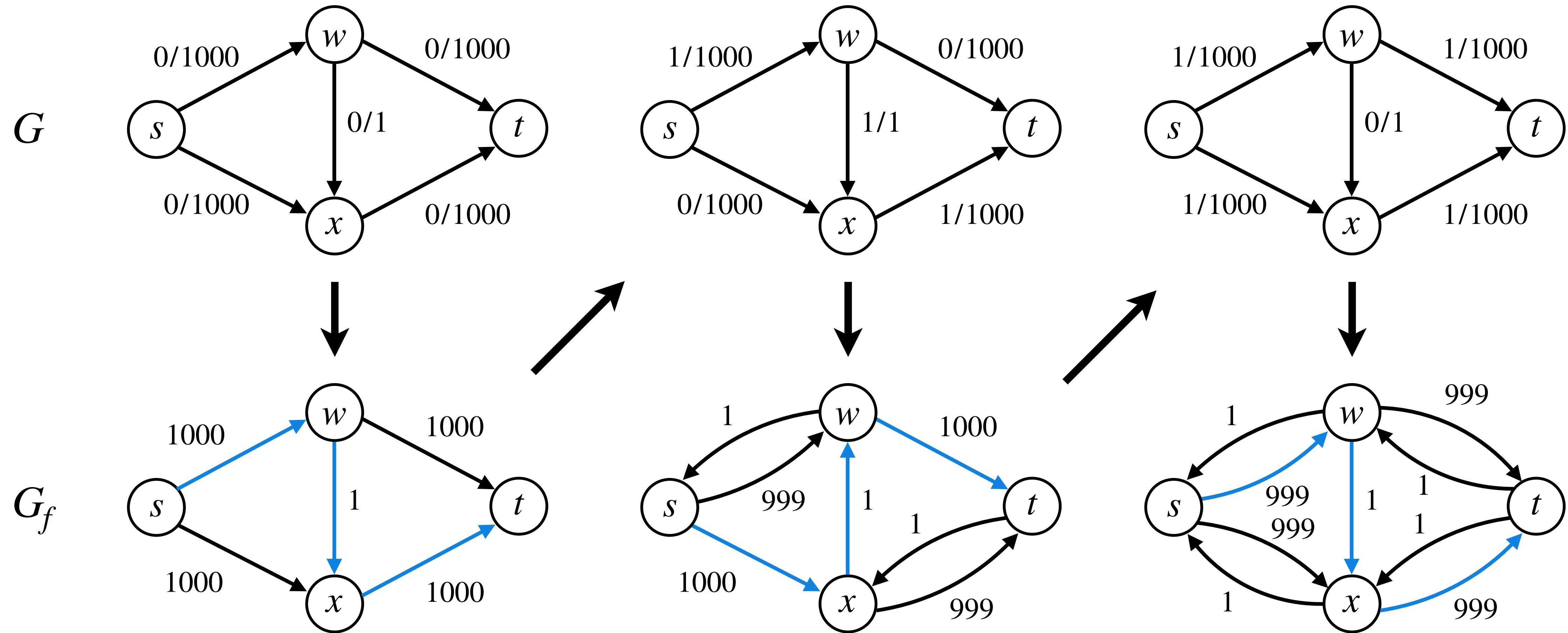
A Bad Case for Ford-Fulkerson



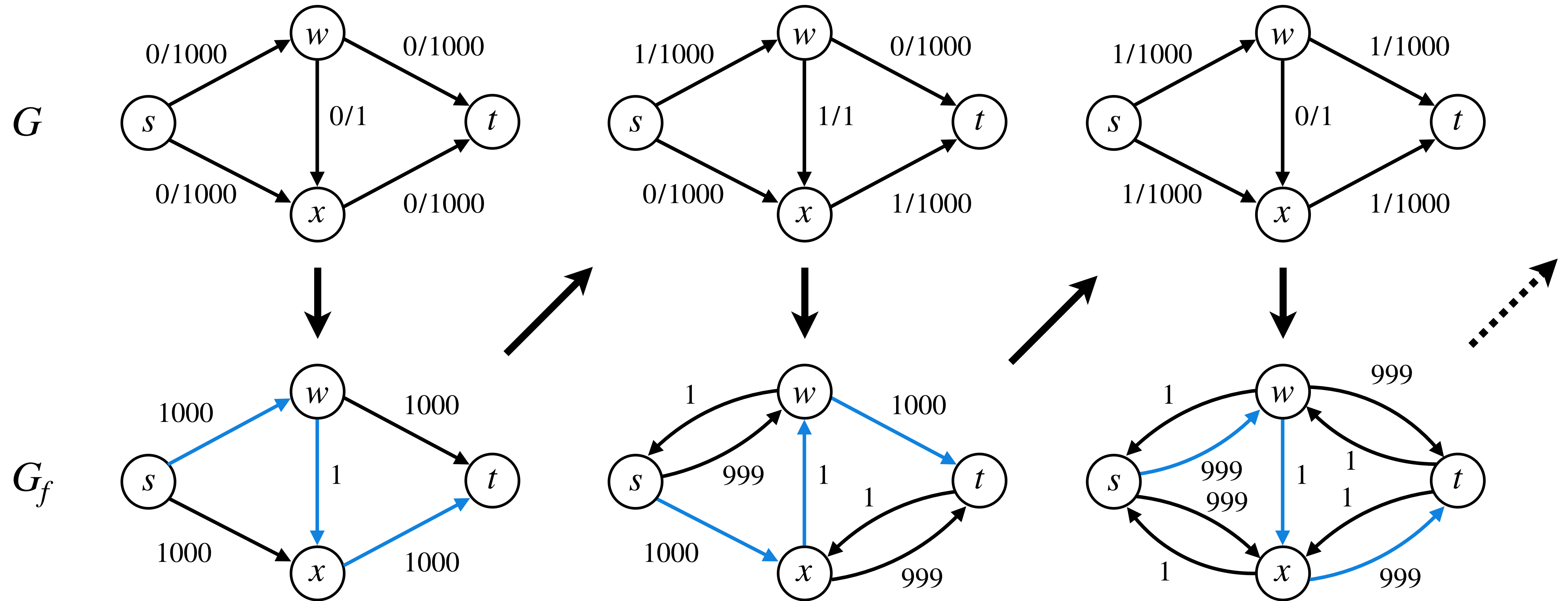
A Bad Case for Ford-Fulkerson



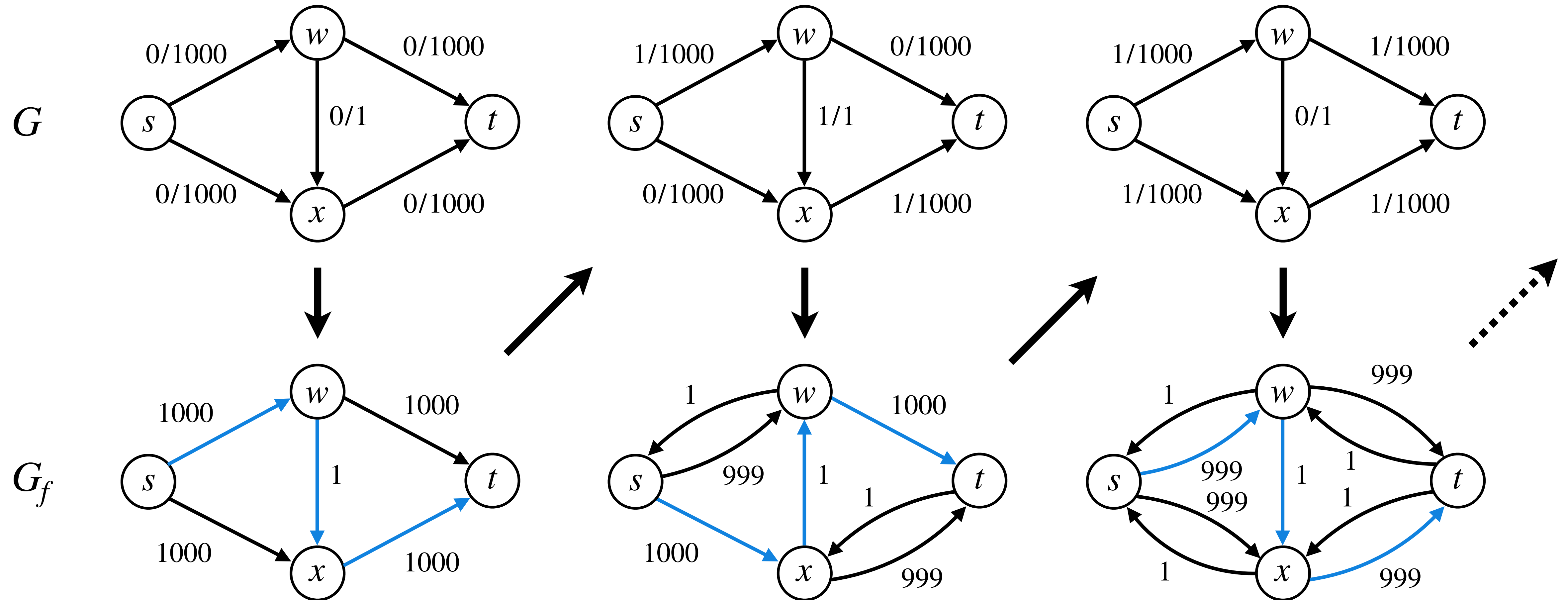
A Bad Case for Ford-Fulkerson



A Bad Case for Ford-Fulkerson



A Bad Case for Ford-Fulkerson



This way will take 2000 iterations, while 2 can do the job.

A Bad Case for Ford-Fulkerson

A Bad Case for Ford-Fulkerson

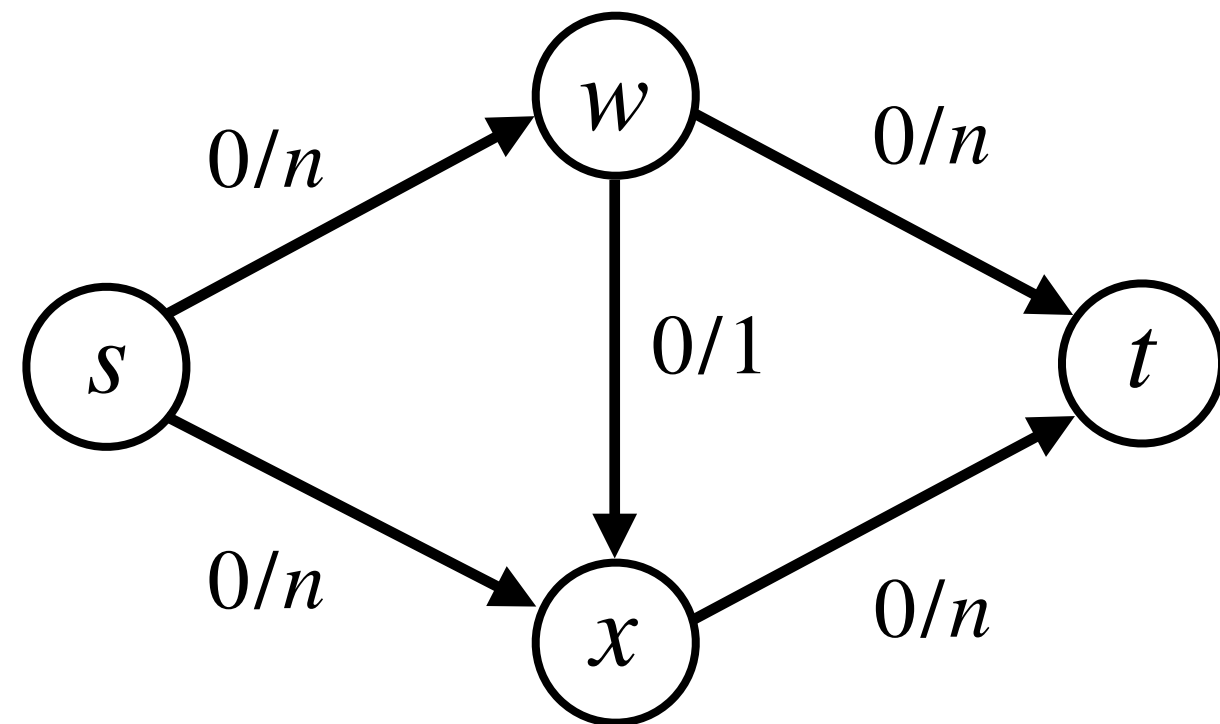
Issue: Runtime of Ford-Fulkerson $\Theta(|E| \cdot |f^*|)$ depends on f and exponential in the length of

A Bad Case for Ford-Fulkerson

Issue: Runtime of Ford-Fulkerson $\Theta(|E| \cdot |f^*|)$ depends on f and exponential in the length of the input.

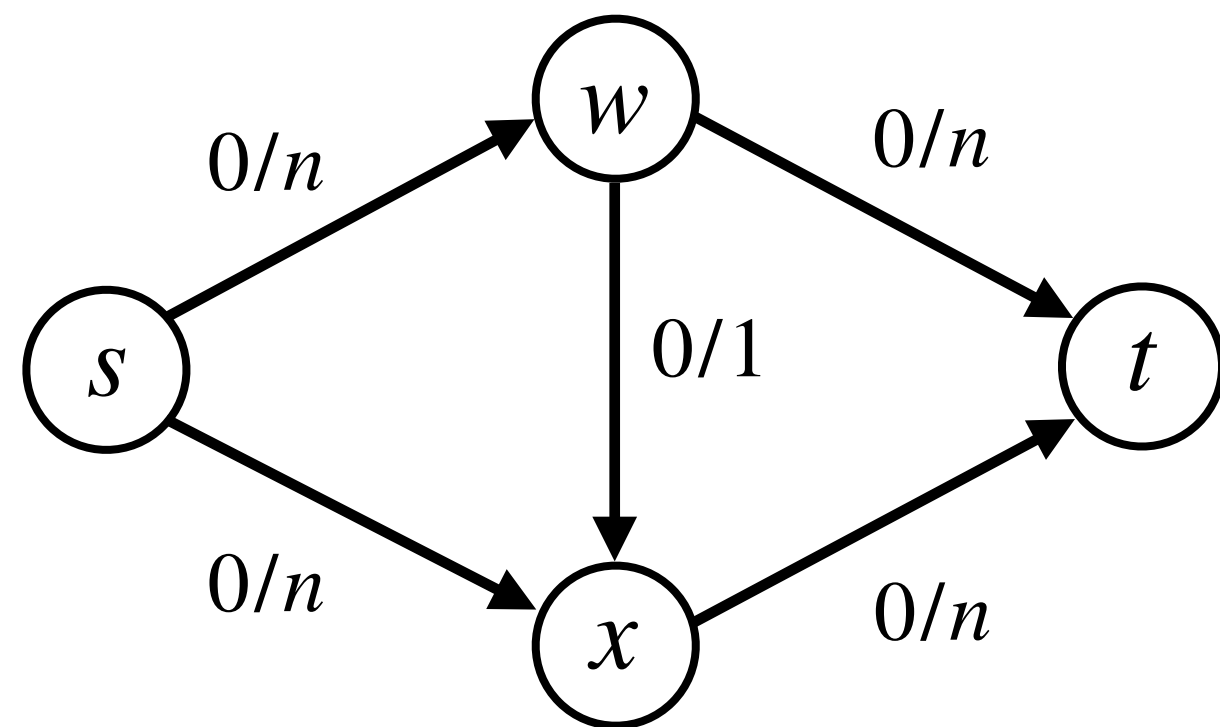
A Bad Case for Ford-Fulkerson

Issue: Runtime of Ford-Fulkerson $\Theta(|E| \cdot |f^*|)$ depends on f and exponential in the length of the input.



A Bad Case for Ford-Fulkerson

Issue: Runtime of Ford-Fulkerson $\Theta(|E| \cdot |f^*|)$ depends on f and exponential in the length of the input.



Capacities are given using $O(\log n)$ bits while Ford-Fulkerson can run for n iterations.

A Bad Case for Ford-Fulkerson

Issue: Runtime of Ford-Fulkerson $\Theta(|E| \cdot |f^*|)$ depends on f and exponential in the length of the input.

A Bad Case for Ford-Fulkerson

Issue: Runtime of Ford-Fulkerson $\Theta(|E| \cdot |f^*|)$ depends on f and exponential in the length of the input. Because we are not choosing augmenting paths carefully.

A Bad Case for Ford-Fulkerson

Issue: Runtime of Ford-Fulkerson $\Theta(|E| \cdot |f^*|)$ depends on f and exponential in the length of the input. Because we are not choosing augmenting paths carefully.

Fix:

A Bad Case for Ford-Fulkerson

Issue: Runtime of Ford-Fulkerson $\Theta(|E| \cdot |f^*|)$ depends on f and exponential in the length of the input. Because we are not choosing augmenting paths carefully.

Fix: The method where we always pick the shortest length augmenting path in G_f is known as

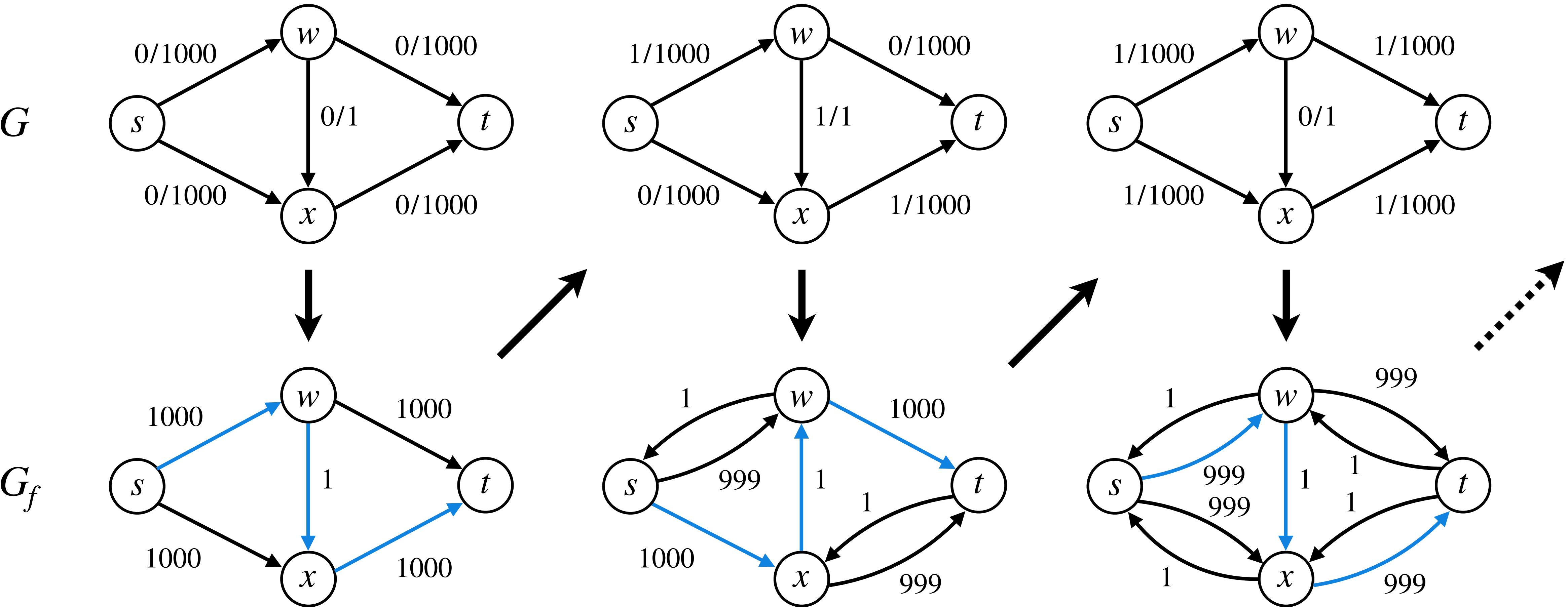
A Bad Case for Ford-Fulkerson

Issue: Runtime of **Ford-Fulkerson** $\Theta(|E| \cdot |f^*|)$ **depends** on f and **exponential** in the length of the input. Because we are not choosing augmenting paths carefully.

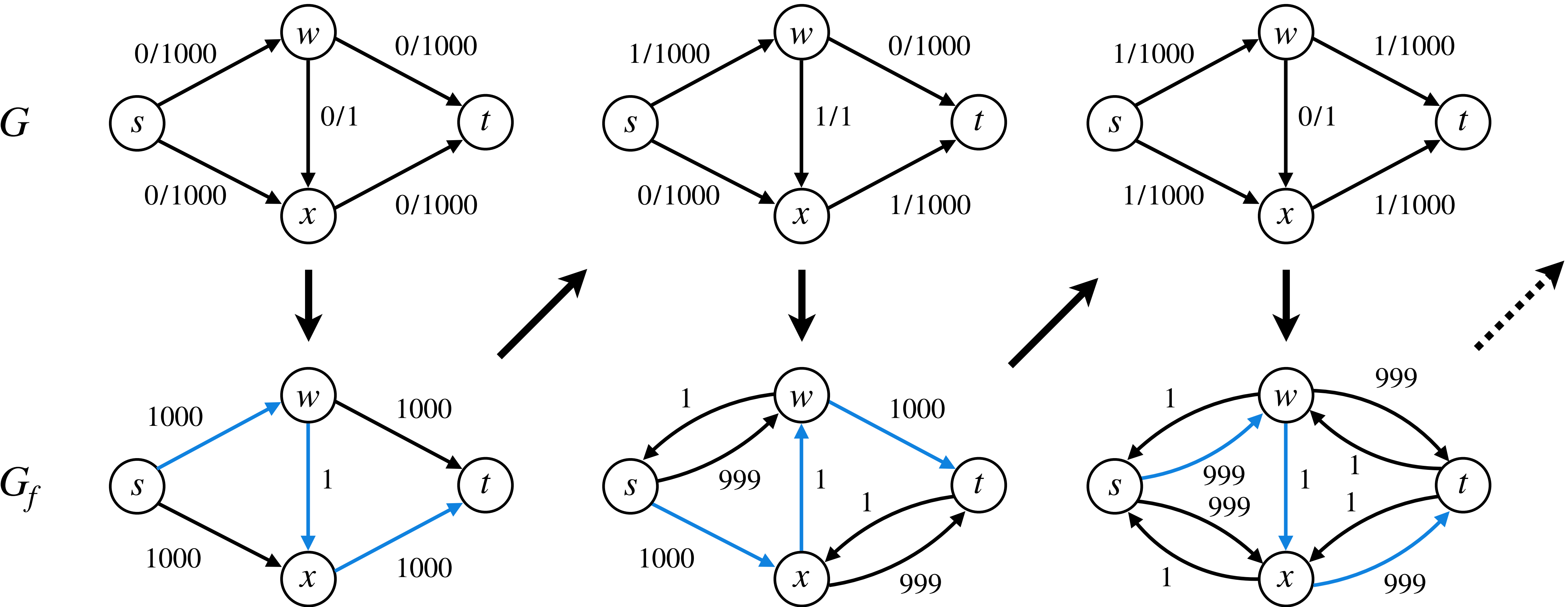
Fix: The method where we always pick the **shortest length** augmenting path in G_f is known as **Edmonds-Karp** method, which has a polynomial runtime that doesn't depend on f .

Edmonds-Karp: Idea

Edmonds-Karp: Idea



Edmonds-Karp: Idea



Notice how (w, x) keeps on disappearing and reappearing in G_f s.

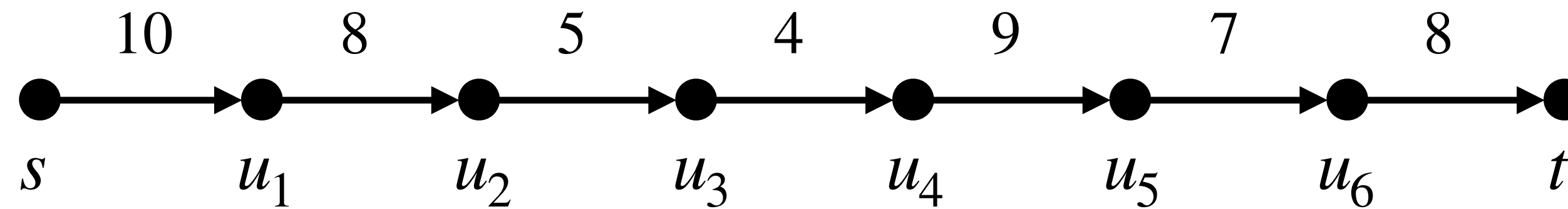
Edmonds-Karp: Idea

Edmonds-Karp: Idea

Suppose the below augmenting path was selected in some iteration in G_f :

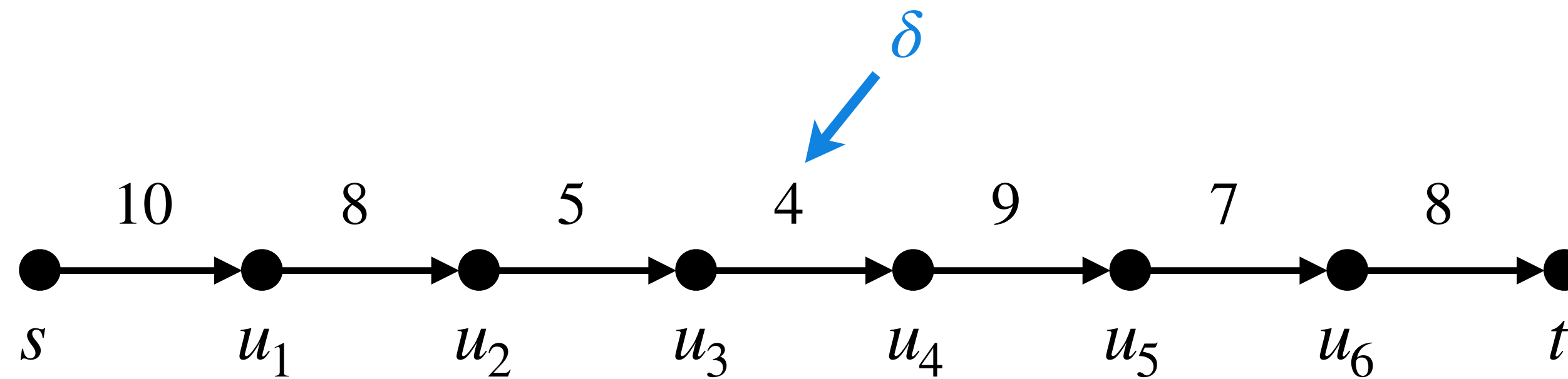
Edmonds-Karp: Idea

Suppose the below augmenting path was selected in some iteration in G_f :



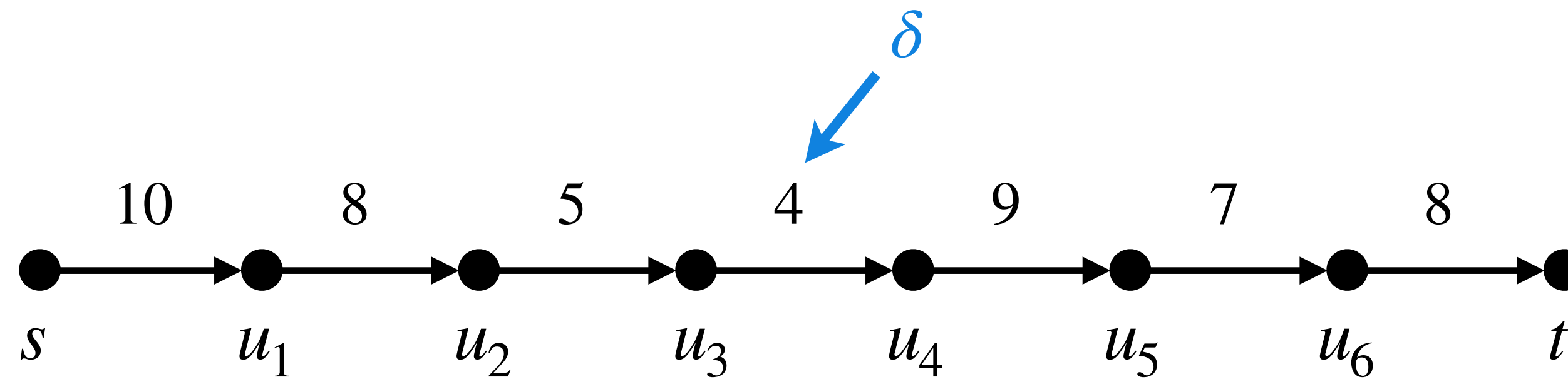
Edmonds-Karp: Idea

Suppose the below augmenting path was selected in some iteration in G_f :



Edmonds-Karp: Idea

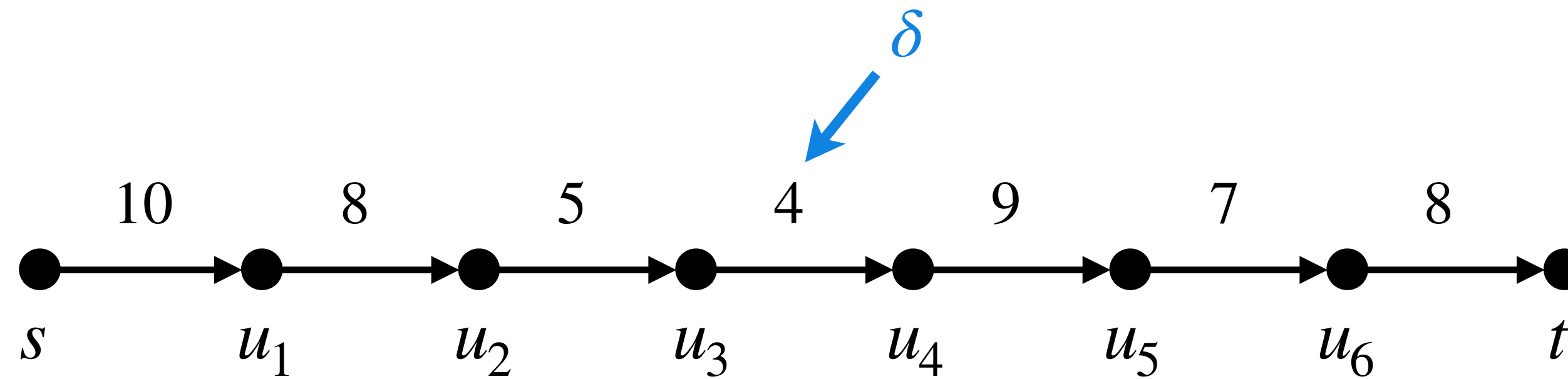
Suppose the below augmenting path was selected in some iteration in G_f :



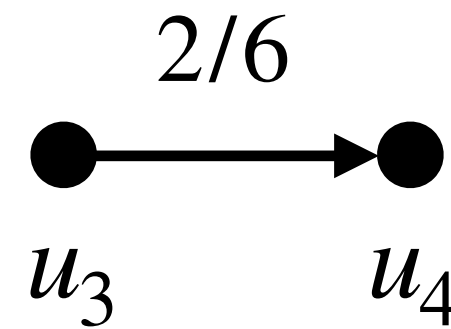
If (u_3, u_4) was a forward edge:

Edmonds-Karp: Idea

Suppose the below augmenting path was selected in some iteration in G_f :



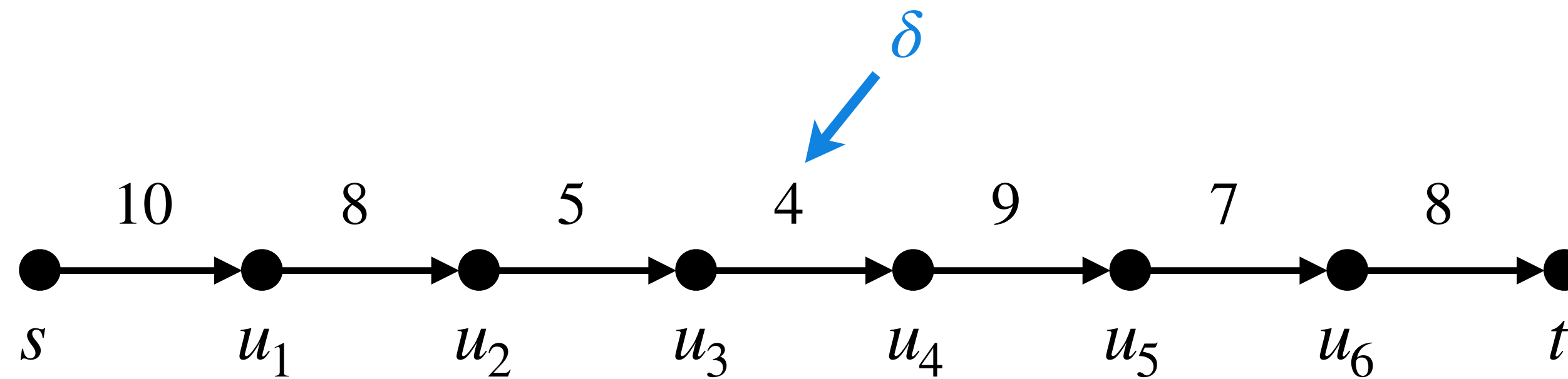
If (u_3, u_4) was a forward edge:



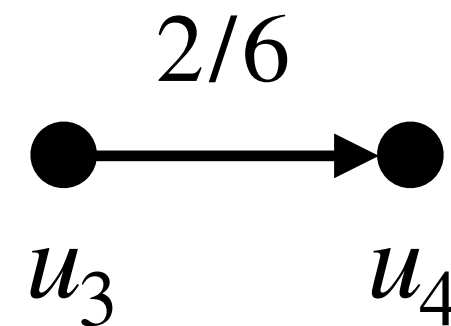
Before augmentation

Edmonds-Karp: Idea

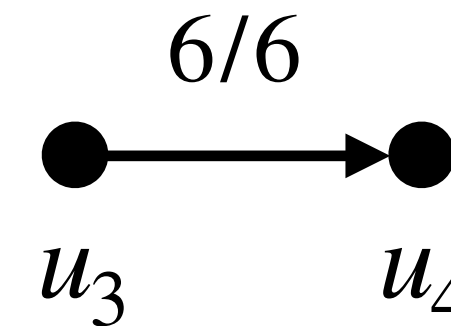
Suppose the below augmenting path was selected in some iteration in G_f :



If (u_3, u_4) was a forward edge:



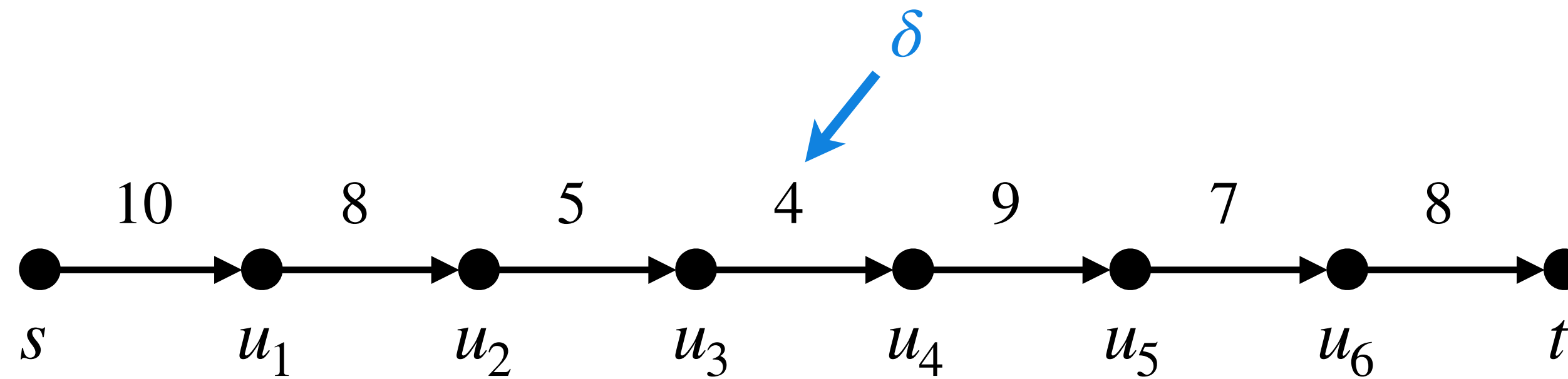
Before augmentation



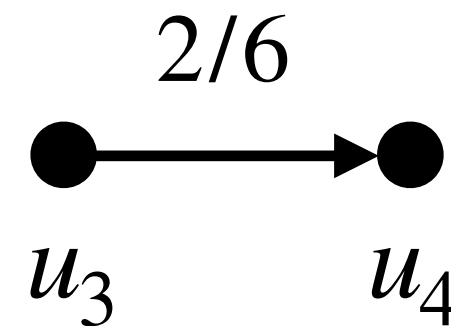
After augmentation

Edmonds-Karp: Idea

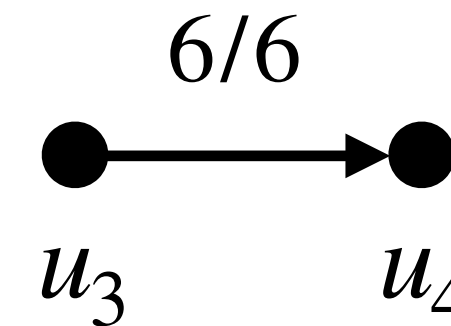
Suppose the below augmenting path was selected in some iteration in G_f :



If (u_3, u_4) was a forward edge:



Before augmentation

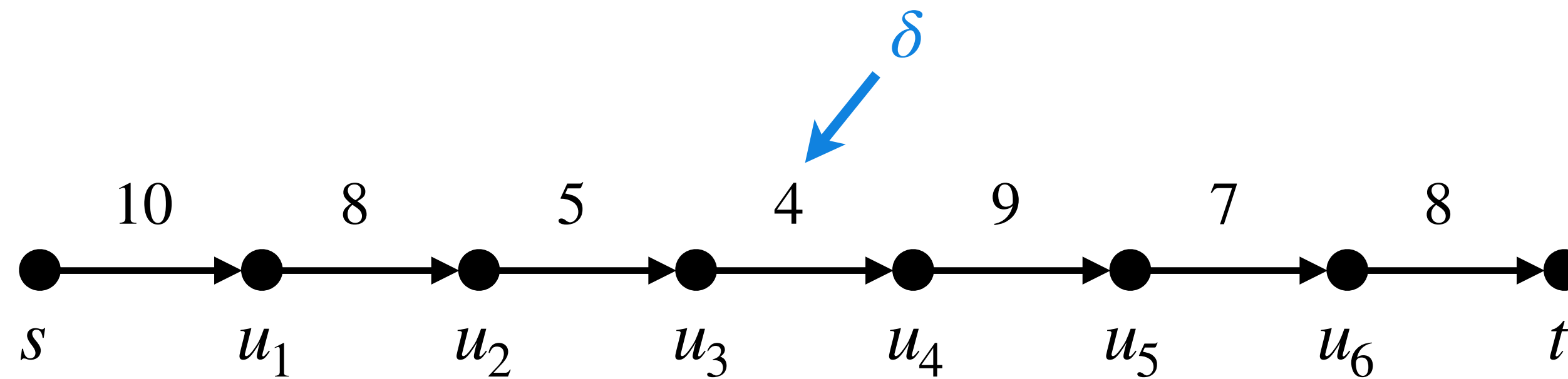


After augmentation

(u_3, u_4) will disappear
in the residual network.

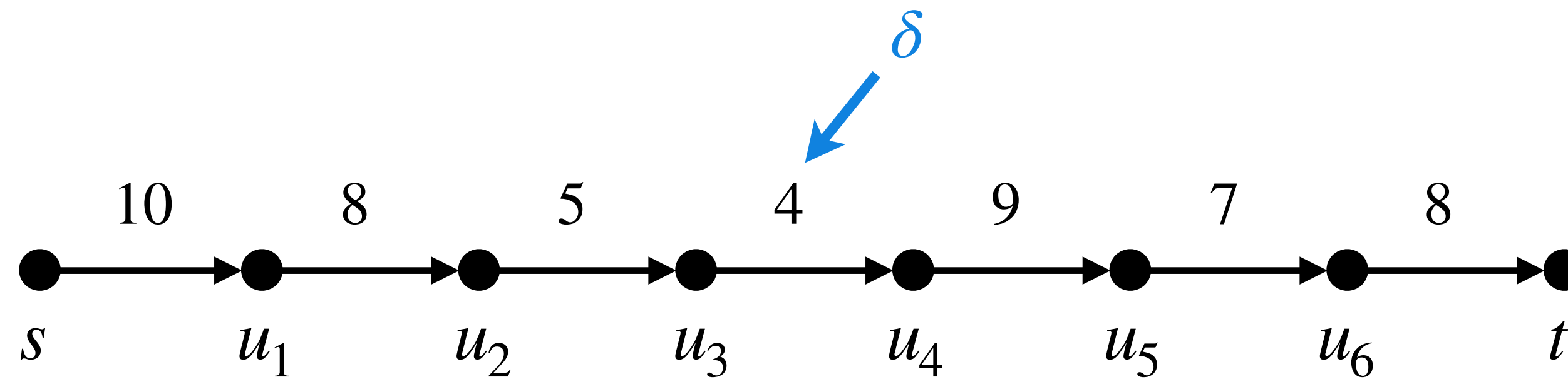
Edmonds-Karp: Idea

Suppose the below augmenting path was selected in some iteration in G_f :



Edmonds-Karp: Idea

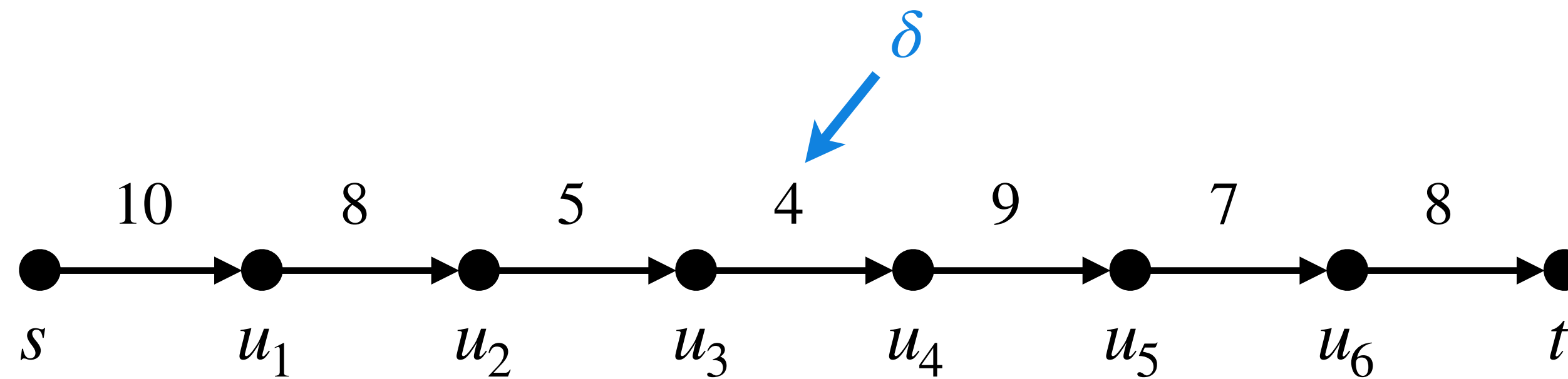
Suppose the below augmenting path was selected in some iteration in G_f :



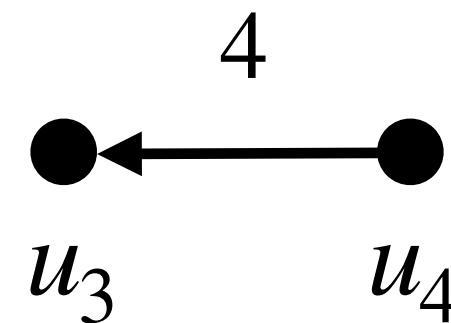
If (u_3, u_4) was a backward edge:

Edmonds-Karp: Idea

Suppose the below augmenting path was selected in some iteration in G_f :



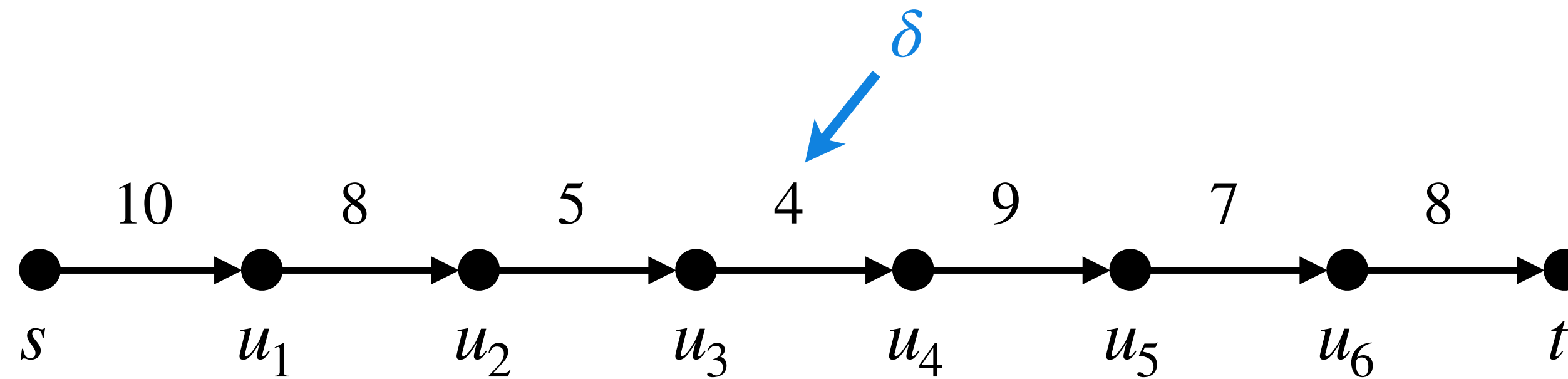
If (u_3, u_4) was a backward edge:



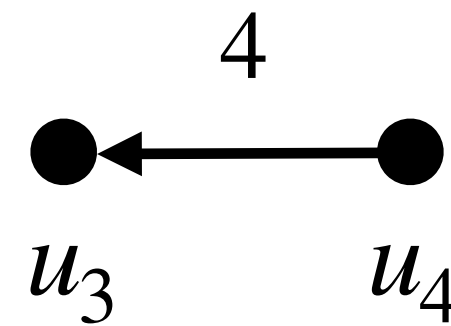
Before augmentation

Edmonds-Karp: Idea

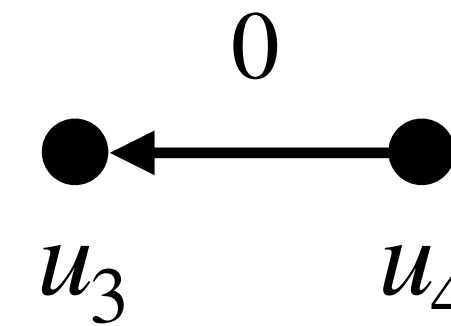
Suppose the below augmenting path was selected in some iteration in G_f :



If (u_3, u_4) was a backward edge:



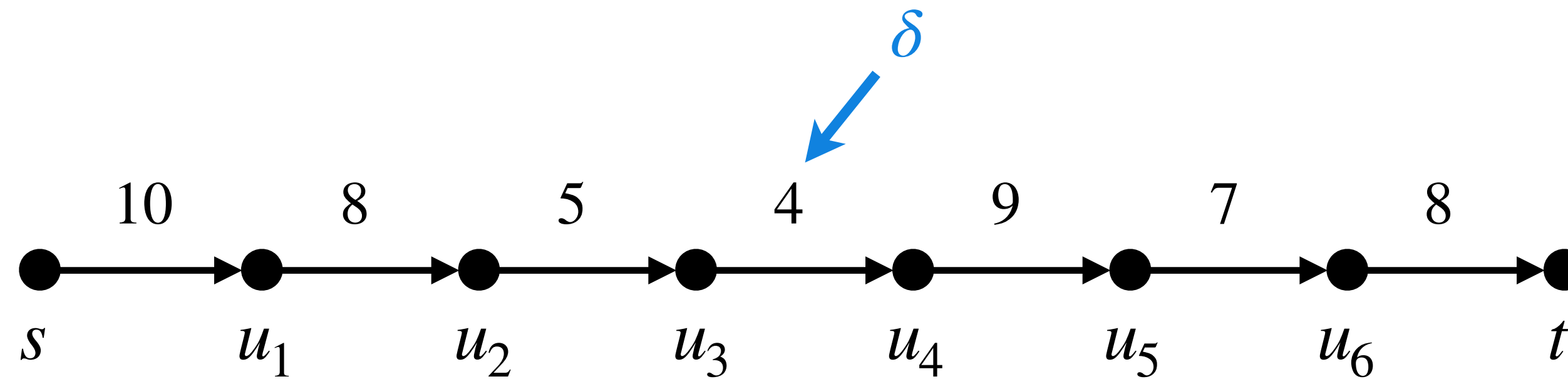
Before augmentation



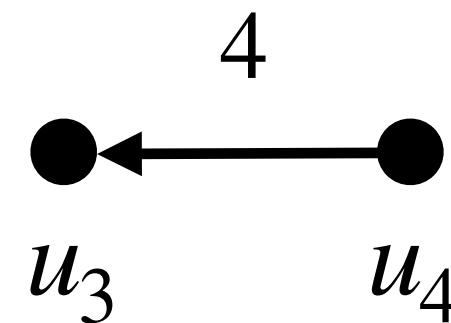
After augmentation

Edmonds-Karp: Idea

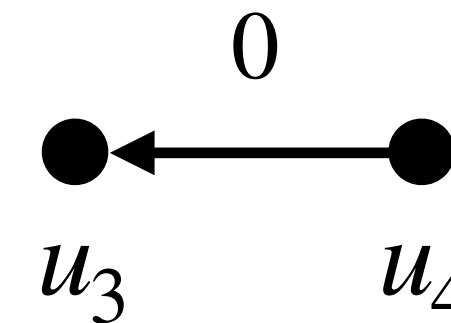
Suppose the below augmenting path was selected in some iteration in G_f :



If (u_3, u_4) was a backward edge:



Before augmentation

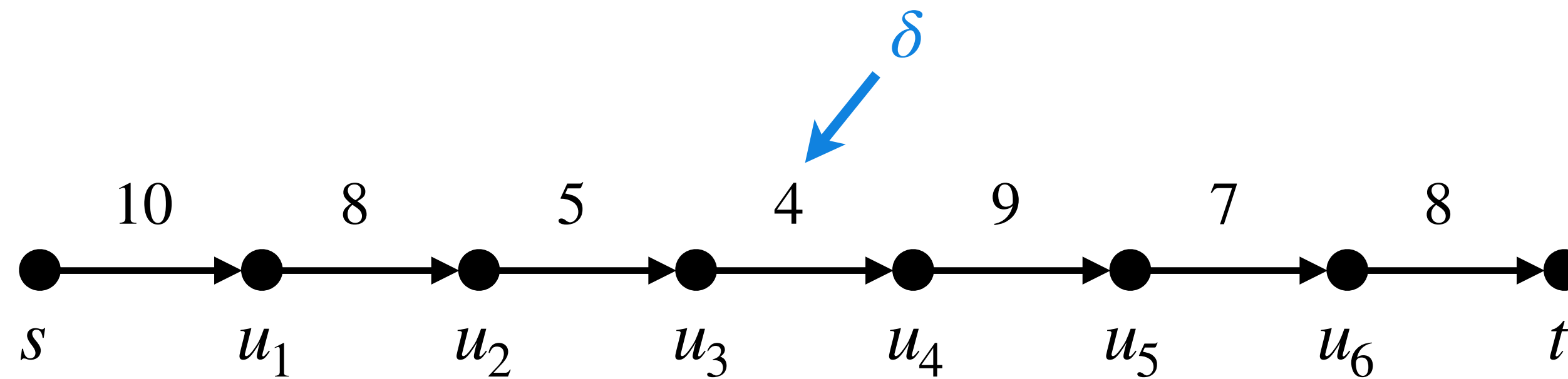


After augmentation

(u_3, u_4) will disappear
in the residual network.

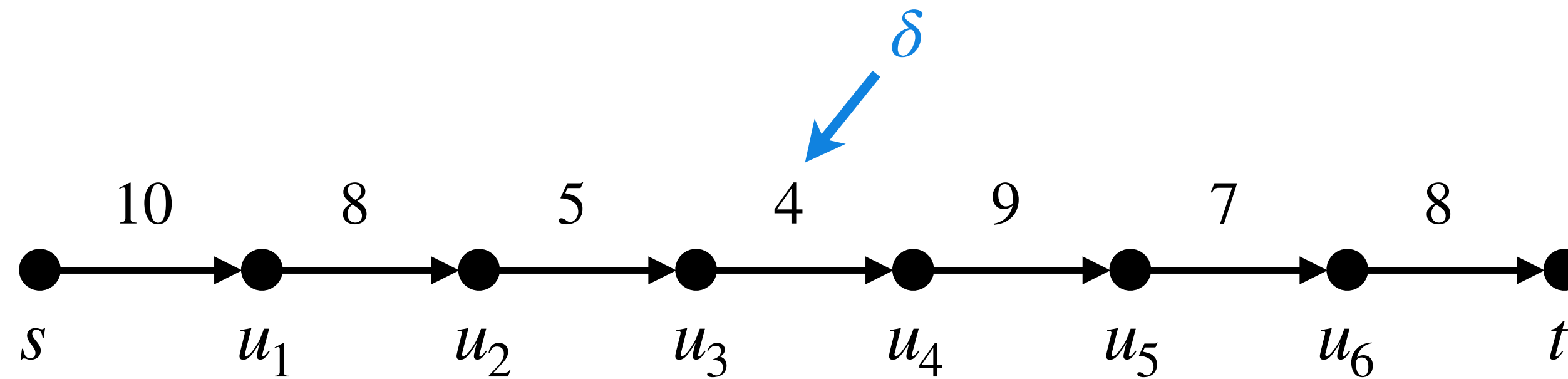
Edmonds-Karp: Idea

Suppose the below augmenting path was selected in some iteration in G_f :



Edmonds-Karp: Idea

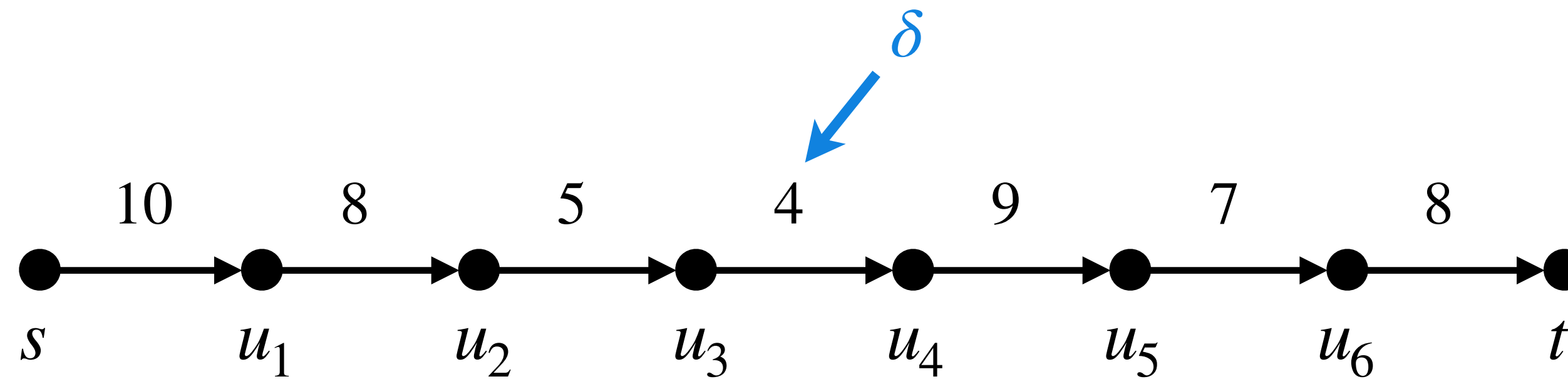
Suppose the below augmenting path was selected in some iteration in G_f :



Observation: In every iteration of **Ford-Fulkerson**, at least one edge will disappear in the next

Edmonds-Karp: Idea

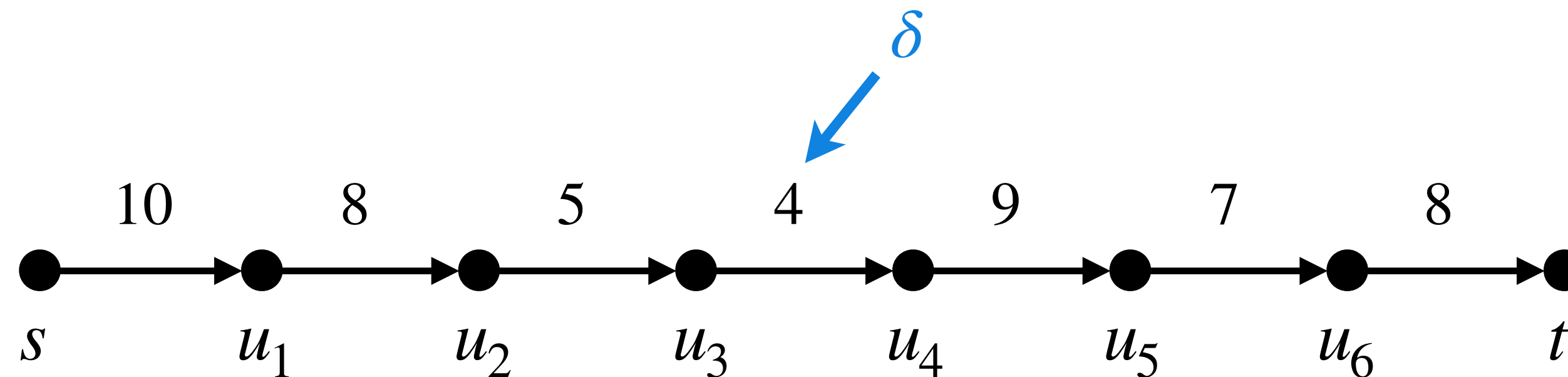
Suppose the below augmenting path was selected in some iteration in G_f :



Observation: In every iteration of **Ford-Fulkerson**, at least one edge will disappear in the next residual network after the flow augmentation.

Edmonds-Karp: Idea

Suppose the below augmenting path was selected in some iteration in G_f :

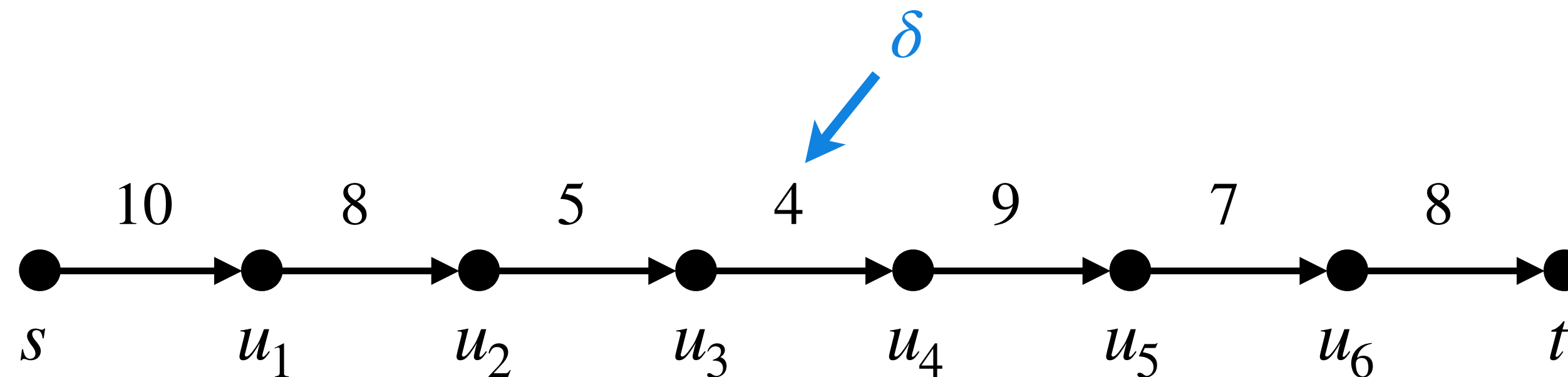


Observation: In every iteration of **Ford-Fulkerson**, at least one edge will disappear in the next residual network after the flow augmentation.

Idea: Picking **shortest length** augmenting paths ensures an edge cannot disappear-reappear **too**

Edmonds-Karp: Idea

Suppose the below augmenting path was selected in some iteration in G_f :



Observation: In every iteration of **Ford-Fulkerson**, at least one edge will disappear in the next residual network after the flow augmentation.

Idea: Picking **shortest length** augmenting paths ensures an edge cannot disappear-reappear **too many times**.